

به نام خدا

نام درس: علوم داده

گزارش: HW1

نام: نیوشا یقینی

شماره دانشجویی: 98522346

تاریخ: آبان/1403

فهرست مطالب

۳	 بخش تئوری
۳	 ۱. سوال شماره ۱
۵	 ۲. سوال شماره ۲
۸	 ۳. سوال شماره ۳
۱۱	 بخش عملی
۱۱	 ۱. سوال شماره ۱
۲۶	 ۲. سوال شماره ۲
۳۴	 ۳. سوال شماره ۳
۵۵	 منابع

بخش تئوری:

۱. سوال شماره ۱:

در مورد Maximum Likelihood Estimation تحقیق کنید و در دو پاراگراف به اختصار توضیح دهید.

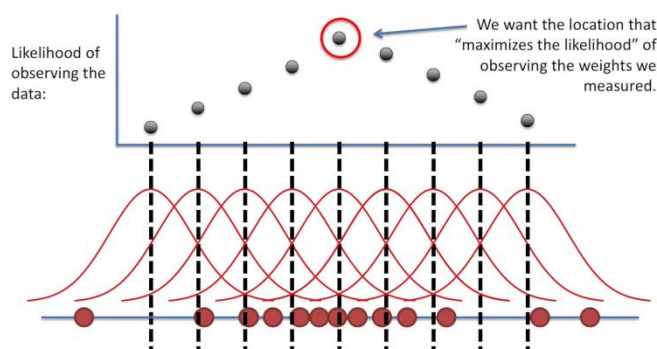
این سوال تئوری می‌باشد.

➤ توضیحات

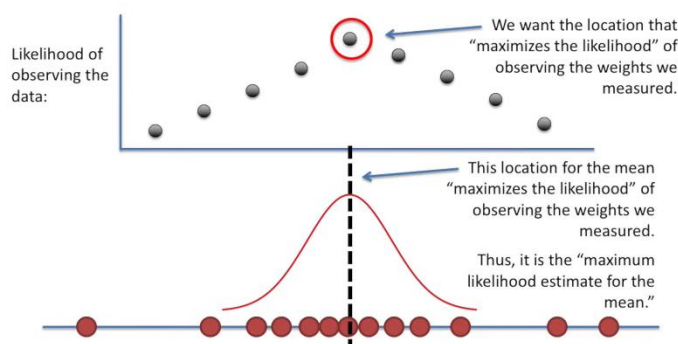
روش تخمین بیشینه احتمال Maximum Likelihood Estimation یا (MLE) یکی از روش‌های آماری برای برآورد پارامترهای یک مدل است. ایده اصلی MLE این است که مقادیری برای پارامترهای مدل پیدا کنیم که بیشترین احتمال را به داده‌های مشاهده‌شده نسبت دهند. فرض کنید داده‌ها از یک توزیع خاص می‌آیند که توسط پارامترهای نامشخصی تعیین شده است. با استفاده از تابع احتمال، MLE این پارامترها را طوری تخمین می‌زند که احتمال (Likelihood) مشاهده داده‌های فعلی به حداکثر برسد.

در MLE، تابع احتمال داده‌ها نسبت به پارامترها حداکثر می‌شود تا تخمین بهینه حاصل شود. این روش در مدل‌های آماری مختلف از جمله رگرسیون و یادگیری ماشین به کار می‌رود، زیرا برای پارامترهای بهینه به ما راه حلی می‌دهد که داده‌ها را به بهترین شکل توصیف می‌کند. از لحاظ ریاضی، این کار از طریق مشتق‌گیری از تابع احتمال و برابر صفر قرار دادن آن انجام می‌شود، که معمولاً منجر به یک معادله بهینه‌سازی می‌شود. در عمل، از MLE به دلیل کارایی و سادگی در تخمین پارامترها بسیار استفاده می‌شود.

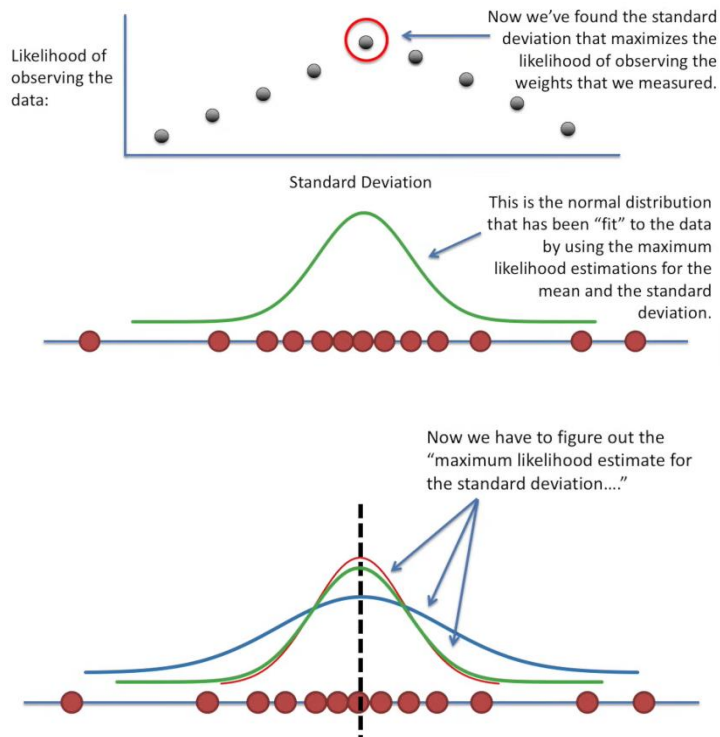
➤ بررسی یک مثال



در این تصویر، هدف پیدا کردن موقعیتی است که احتمال مشاهده داده‌ها را به حداکثر برساند. نقاط قرمز نشان‌دهنده داده‌های مشاهده‌شده و توزیع‌ها نشان‌دهنده احتمال مشاهده این داده‌ها در مکان‌های مختلف هستند. هر کدام از توزیع‌ها نشان‌دهنده تلاش برای یافتن پارامترهایی است که مشاهده داده‌ها در آن نقطه را به حداکثر برسانند.



در این مرحله، مکان میانگین به گونه‌ای انتخاب شده که احتمال مشاهده داده‌ها را به حداکثر برساند. این نقطه همان تخمین بیشینه احتمال برای میانگین است. میانگین جایی قرار گرفته که احتمال مشاهده داده‌ها بیشترین مقدار خود را دارد و داده‌ها به بهترین شکل با توزیع نرمال مطابقت دارند.



اکنون باید مقدار انحراف معیار را طوری تنظیم کنیم که احتمال مشاهده داده‌ها بهینه شود. چند توزیع مختلف با انحراف معیارهای مختلف نمایش داده شده‌اند و هدف این است که انحراف معیاری انتخاب شود که به بهترین شکل داده‌ها را پوشش دهد.

در نهایت، انحراف معیاری پیدا شده که حداکثر احتمال مشاهده داده‌ها را تضمین می‌کند. این توزیع نرمال با استفاده از تخمین‌های بیشینه احتمال برای میانگین و انحراف معیار به داده‌ها "فیت" شده است.

۲. سوال شماره ۲:

فرض کنید مقادیر $x_1, x_2, \dots, x_n$ از یک توزیع احتمال با چگالی زیر نمونه‌برداری شدند. با استفاده از Log-Likelihood پارامتر θ (theta) را بر حسب n پیدا کنید.

$$f(x|\theta) = \begin{cases} -\theta x_i^{\theta-1} & , \quad x < 0 \\ 2^\theta & , \quad x \geq 0 \end{cases}$$

این سوال تئوری می‌باشد.

➤ توضیحات

توزیع احتمال و چگالی (Probability Distribution and Density):

توزیع احتمال (Probability Distribution) نحوه پراکندگی یا پخش داده‌ها در دامنه ممکن مقادیر را نشان می‌دهد. یعنی به مقادیر مختلف از یک متغیر تصادفی، احتمال تخصیص می‌دهد، این تخصیص در داده‌های پیوسته با تابع چگالی احتمال توصیف می‌شود، **تابع چگالی احتمال (PDF)** می‌گوید که در هر نقطه خاص، چقدر احتمال مشاهده متغیر تصادفی نزدیک به آن نقطه وجود دارد. با استفاده از PDF می‌توان احتمال اینکه یک متغیر تصادفی در یک بازه خاص باشد را محاسبه کرد. در این سوال، یک توزیع احتمال داده شده که فقط در مقادیر $x < 0$ دارای مقدار چگالی احتمال است و در مقادیر $x \geq 0$ صفر می‌شود، یعنی داده‌ها فقط در دامنه منفی دارای چگالی غیر صفر هستند. این نشان می‌دهد که احتمال مشاهده مقادیر مثبت یا صفر برای این توزیع، صفر است.

(Maximum Likelihood Estimation - MLE):

MLE پارامتری از مدل را تخمین می‌زند که احتمال مشاهده داده‌ها را به بیشترین مقدار می‌رساند. درواقع آن‌ها را به بهترین شکل با مدل تطبیق می‌دهد. همچنین MLE برابر است با حاصل ضرب چگالی احتمال هر مقدار x_i از داده‌ها.

تابع Log-Likelihood:

تابع درست‌نمایی (Likelihood) نمایانگر احتمال وقوع داده‌های مشاهده‌شده بر اساس پارامترهای مدل است. برای ساده‌تر کردن محاسبات، اغلب لگاریتم این تابع را می‌گیریم، چراکه حاصل ضرب را به حاصل جمع تبدیل می‌کند.

➤ مراحل حل سوال

$$L(\theta) = \prod_{i=1}^n f(x_i|\theta) \quad \text{1. تعریف تابع MLE}$$

$$\ln L(\mu) = \sum_{i=1}^n \ln f(x_i|\mu, \sigma) \quad \text{2. گرفتن لگاریتم طبیعی (Log-Likelihood)}$$

$$\frac{d}{d\theta} \ln L(\theta) = 0 \quad \text{3. پیدا کردن MLE با مشتق‌گیری}$$

$$f(x|\theta) = \begin{cases} -\frac{\theta x_i^{\theta-1}}{\Gamma(\theta)} & , x < 0 \\ 0 & , x \geq 0 \end{cases} \quad (\text{PDF})$$

$$L(\theta) = \prod_{i=1}^n f(x_i|\theta) \quad \text{تعریف MLE}$$

$$\rightarrow L(\theta) = \prod_{i=1}^n \left(-\frac{\theta x_i^{\theta-1}}{\Gamma(\theta)} \right) = \left(-\frac{\theta}{\Gamma(\theta)} \right)^n \prod_{i=1}^n (x_i^{\theta-1}) = \left(-\frac{\theta^n}{\Gamma(\theta)^n} \right) \prod_{i=1}^n (x_i^{\theta-1})$$

* مقدار $\left(-\frac{\theta}{\Gamma(\theta)} \right)$ به عنوان ضریب با n های مختلف تغییر می کند و فقط در اینجا

n بار در خودش ضرب شده است، بنابراین این را $\prod$ می نویسند و این را داریم

$$\ln L(\theta) \Rightarrow \ln \left(\left(-\frac{\theta}{\Gamma(\theta)} \right)^n \prod_{i=1}^n (x_i^{\theta-1}) \right) = \quad \text{log-likelihood} \quad (2)$$

$$n \ln \left(-\frac{\theta}{\Gamma(\theta)} \right) + (\theta-1) \sum_{i=1}^n \ln x_i \Rightarrow$$

در لگاریتم لیکی، حاصل ضرب به حاصل جمع تبدیل می شود.

همین توان به ضریب تبدیل می شود و ضرب به جمع.

از لگاریتم لیکی استفاده کردیم و به این پایه توزیع های احتمالی که از آن به دست می آید است. (طالعات)

فصلی متداول تر است در این مسئله

$$\Rightarrow \ln L(\theta) = n \ln(-\theta) - n \theta \ln \Gamma + (\theta-1) \sum_{i=1}^n \ln x_i$$

MLE: Maximum Likelihood Estimation

$$\frac{d}{d\theta} \ln L(\theta) = 0$$

$$\frac{d}{d\theta} \left(\underbrace{n \ln(-\theta)}_{(1)} - \underbrace{n\theta \ln r}_{(2)} + (\theta-1) \underbrace{\sum_{i=1}^n \ln x_i}_{(3)} \right) =$$

$$(1): \frac{d}{d\theta} (n \ln(-\theta)) = n \cdot \frac{d}{d\theta} \ln(-\theta) = n \cdot \left(\frac{1}{-\theta} \right) \cdot \frac{d}{d\theta} (-\theta)$$

$$= n \cdot \frac{1}{-\theta} \cdot (-1) = + \frac{n}{\theta}$$

$$(2): \frac{d}{d\theta} (-n\theta \ln r) = -n \ln r$$

$$(3): \frac{d}{d\theta} \left((\theta-1) \sum_{i=1}^n \ln x_i \right) = \frac{d}{d\theta} \left(\theta \sum_{i=1}^n \ln x_i - \sum_{i=1}^n \ln x_i \right) = \sum_{i=1}^n \ln x_i$$

$$\text{total} \Rightarrow \frac{d}{d\theta} L(\theta) = \frac{n}{\theta} - n \ln r + \sum_{i=1}^n \ln x_i = 0$$

$$\Rightarrow \frac{n}{\theta} = n \ln r - \sum_{i=1}^n \ln x_i \Rightarrow \theta = \frac{n}{n \ln r - \sum_{i=1}^n \ln x_i}$$

$$\hat{\theta} = \frac{1}{\ln r - \frac{1}{n} \sum_{i=1}^n \ln x_i}$$

۳. سوال شماره ۳:

فردی ۱۶ بار تاس انداخته و نتیجه به این شرح است: ۱۵ بار عدد ۱ و تنها ۱ بار عدد ۰ ظاهر شده است. این تاس یک تاس غیرمنصفانه است که احتمال آمدن عدد ۱ با مقدار P تعیین می‌شود. با توجه به این داده‌ها:

الف) تخمین بزنید که احتمال آمدن عدد ۱ با استفاده از روش Maximum Likelihood چقدر است؟ توضیح دهید که چرا این روش در این مثال استفاده می‌شود و چگونه می‌توان نتیجه را بر اساس آن تفسیر کرد.

ب) اگر داده‌های شما فقط شامل ۸ بار پرتاب بود و همچنان ۷ بار عدد ۱ و ۱ بار عدد ۰ ظاهر می‌شد، تخمین شما برای احتمال P چگونه تغییر می‌کرد؟

مقایسه‌ای بین این دو مجموعه داده (۱۶ بار و ۸ بار) انجام دهید و در مورد تأثیر تعداد داده‌های مشاهده شده بر روی دقت تخمین صحبت کنید.

این سوال تئوری می‌باشد.

➤ توضیحات

مفهوم احتمال و متغیر تصادفی:

در این سوال، احتمال P نشان‌دهنده‌ی احتمال آمدن عدد ۱ در پرتاب یک تاس غیرمنصفانه است. این احتمال بین ۰ و ۱ قرار دارد، و هدف ما تخمین مقدار P بر اساس داده‌های مشاهده شده است. در اینجا، هر بار پرتاب تاس یک **آزمایش برنولی** محسوب می‌شود که دو نتیجه ممکن دارد: یا عدد ۱ ظاهر می‌شود یا عدد ۰.

(Maximum Likelihood Estimation - MLE):

MLE روشی است که به ما کمک می‌کند مقدار پارامترهای مدل را طوری تخمین بزنیم که احتمال وقوع داده‌های مشاهده شده بیشترین مقدار ممکن را داشته باشد. در این سوال، هدف ما یافتن مقداری از P است که بیشترین **درست‌نمایی** را برای داده‌های مشاهده شده تولید کند.

تابع Likelihood:

برای یک توزیع برنولی با احتمال P ، اگر n بار آزمایش انجام شود و k بار عدد ۱ ظاهر شود، **MLE** به صورت زیر تعریف می‌شود:

$$L(p) = p^k \cdot (1 - p)^{n-k}$$

این تابع بیانگر احتمال به دست آمدن داده‌های مشاهده شده (عدد ۱ به تعداد k بار و عدد ۰ به تعداد $n-k$ بار) بر اساس مقدار P است.

یافتن مقدار بیشینه درست‌نمایی برای P:

برای یافتن مقدار P که تابع Likelihood را به بیشترین مقدار خود برساند، می‌توانیم ابتدا **لگاریتم طبیعی تابع MLE** را محاسبه کرده و سپس با **مشتق‌گیری** از آن نسبت به P و برابر قرار دادن آن با صفر، مقدار بیشینه P را پیدا کنیم. تابع لگاریتمی یک **تابع یکنواخت افزایشی** است؛ یعنی اگر مقدار ورودی تابع زیاد شود، مقدار خروجی تابع نیز زیاد می‌شود و برعکس. به زبان ریاضی، اگر $y < x$ باشد، آنگاه $\ln(y) < \ln(x)$ نیز خواهد بود.

بنابراین، زمانی که می‌خواهیم تابعی مانند $L(p)$ که همان تابع MLE است را **بیشینه‌سازی** کنیم، می‌توانیم به جای بیشینه‌سازی خود تابع **لگاریتم تابع**، یعنی $\ln(L(p))$ را بیشینه‌سازی کنیم.

در این حالت، تخمین P به شرح زیر است:

نتیجه گرفتن لگاریتم طبیعی:

$$\ln(L(p)) = k \cdot \ln(p) + (n - k) \cdot \ln(1 - p)$$

مقدار بیشینه:

$$\hat{p} = \frac{k}{n}$$

➤ حل سوال

بخش (الف):

در اینجا داده‌ها نشان می‌دهند که از ۱۶ بار پرتاب، ۱۵ بار عدد ۱ ظاهر شده است. بنابراین $n = 16$ و $k = 15$ با استفاده از فرمول MLE:

$$\hat{p} = \frac{k}{n} = \frac{15}{16} = 0.9375$$

بنابراین، تخمین احتمال آمدن عدد ۱ با استفاده از روش MLE برابر با ۰.۹۳۷۵ است.

دلیل استفاده از MLE:

در این سوال، نتیجه هر پرتاب تاس یا عدد ۱ است یا عدد ۰، که هر دو مقدار گسسته هستند و تنها دو حالت ممکن دارند. بنابراین، داده‌ها از توزیع برنولی پیروی می‌کنند و برای چنین توزیعی، روش **بیشینه‌سازی درست‌نمایی (MLE)** به‌خوبی کار می‌کند.

استفاده از روش MLE به ما کمک می‌کند تا با استفاده از داده‌های مشاهده شده، بهترین تخمین را برای احتمال p به دست آوریم. در این روش، تابع درست‌نمایی بر اساس داده‌ها ساخته شده (بطور خاص برای داده‌های گسسته طراحی شده) و با بیشینه‌سازی آن، مقدار بهینه برای p پیدا می‌شود. این تابع به ما امکان می‌دهد که احتمال مشاهده داده‌ها را با توجه به مقدار p مدل کنیم.

از آنجا که هر پرتاب تاس مستقل از پرتاب‌های دیگر است، استفاده از توزیع برنولی و روش MLE در اینجا دقیق و مناسب است. تخمین‌گر

$$\hat{p} = \frac{k}{n}$$

در این روش بهترین تخمین برای احتمال ظاهر شدن عدد ۱ خواهد بود.

در نهایت، اگر داده‌ها پیوسته بودند (مثلاً اندازه‌گیری طول یا وزن)، نیاز به توزیع‌های پیوسته‌ای مانند نرمال یا یکنواخت داشتیم و در نتیجه، تابع درست‌نمایی و فرمول‌های متفاوتی به کار می‌رفتند.

بخش (ب):

در اینجا، اگر تعداد داده‌ها به ۸ بار پرتاب کاهش یابد و همچنان ۷ بار عدد ۱ و ۱ بار عدد ۰ مشاهده شود، در این صورت $n = 8$ و $k = 7$. بنابراین:

$$\hat{p} = \frac{k}{n} = \frac{7}{8} = 0.875$$

در این حالت، تخمین احتمال p برابر با ۰.۸۷۵ می‌شود.

بخش (ج): مقایسه دقت تخمین‌ها

در این بخش باید مقایسه‌ای بین دو مجموعه داده (۱۶ بار و ۸ بار) انجام دهیم و تاثیر تعداد داده‌ها را بررسی کنیم.

هرچه تعداد داده‌ها بیشتر باشد، تخمین دقیق‌تری از پارامتر p بدست می‌آوریم. در واقع، نمونه‌های بیشتر باعث می‌شود که تخمین به مقدار واقعی p نزدیک‌تر شود، زیرا **واریانس تخمین** با افزایش تعداد داده‌ها کاهش می‌یابد.

در این مثال، با داده‌های ۱۶ بار پرتاب، تخمین $p = 0.9375$ بدست آمد که از لحاظ آماری دقیق‌تر از تخمین ۸ بار پرتاب با مقدار $p = 0.875$ است.

بخش عملی:

۱. سوال شماره ۱:

در این سوال دیتای sample.csv در اختیار شما قرار داده شده است. بررسی‌های مورد نیاز روی دیتا را با توجه به نوت بوک انجام دهید.

این سوال عملی می‌باشد.

➤ توضیحات

نمودار KDE یا Kernel Density Estimation روشی برای تخمین غیرپارامتری تابع چگالی احتمال یک متغیر تصادفی است. به عبارت ساده‌تر KDE برای نمایش توزیع داده‌ها به صورت بصری استفاده می‌شود و می‌تواند به درک بهتر الگوها و روندهای موجود در داده‌ها کمک کند.

در این روش، به جای استفاده از هیستوگرام که داده‌ها را به دسته‌هایی تقسیم می‌کند، از یک هسته (kernel) یا تابع ریاضی خاصی برای تخمین پیوسته‌ی توزیع استفاده می‌شود. این هسته‌ها روی نقاط داده اعمال می‌شوند و مجموع آن‌ها توزیع پیوسته‌ای از داده‌ها را نشان می‌دهد.

تابع KDE از یک تابع ریاضی به نام تابع هسته یا Kernel برای تخمین چگالی احتمال استفاده می‌کند. این تابع هسته معمولاً یک تابع پیوسته و صاف است که بر روی هر یک از نقاط داده اعمال می‌شود تا شکل کلی توزیع را به صورت یک منحنی پیوسته نشان دهد.

یکی از هسته‌های متداول که در KDE استفاده می‌شود، هسته گاوسی (Gaussian Kernel) است.

انتخاب تابع هسته و پهنای باند در دقت تخمین KDE بسیار مهم است و می‌تواند با توجه به نوع داده‌ها تنظیم شود.

استفاده‌های اصلی نمودار KDE شامل موارد زیر است:

1. نمایش توزیع داده‌ها: با استفاده از KDE می‌توان به راحتی به شکلی پیوسته و روان، توزیع داده‌ها را مشاهده کرد.
2. بررسی فرضیات مدل‌های پارامتری: این نمودار به بررسی تطابق داده‌ها با مدل‌های پارامتری کمک می‌کند.
3. کشف الگوها و روندها: با استفاده از این نمودار می‌توان الگوها و روندهای احتمالی را در داده‌ها کشف کرد.

در کل، KDE ابزار قدرتمندی برای بررسی و تحلیل داده‌ها به صورت بصری و بدون نیاز به تعیین پارامترهای مشخص است، که می‌تواند در فهم بهتر داده‌ها و تصمیم‌گیری‌های آماری کمک‌کننده باشد.

➤ ساختار کد

1. خواندن دیتا بیس و دیدن هدر ها:

```
df = pd.read_csv('./sample.csv')
df.head()
```

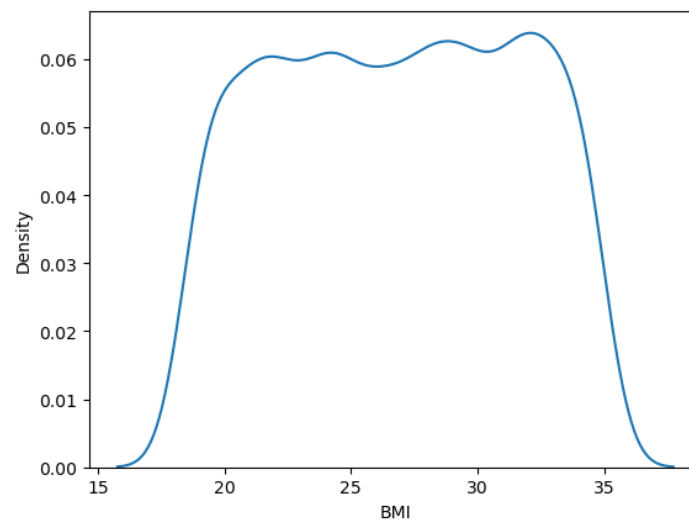
	ID	Exercise	Calories Burn	Dream Weight	Actual Weight	Age	Gender	Duration	Heart Rate	BMI	Weather Conditions	Exercise Intensity
0	1	Exercise 2	286.959851	91.892531	96.301115	45	Male	37	170	29.426275	Rainy	5
1	2	Exercise 7	343.453036	64.165097	61.104668	25	Male	43	142	21.286346	Rainy	5
2	3	Exercise 4	261.223465	70.846224	71.766724	20	Male	20	148	27.899592	Cloudy	4
3	4	Exercise 5	127.183858	79.477008	82.984456	33	Male	39	170	33.729552	Sunny	10
4	5	Exercise 10	416.318374	89.960226	85.643174	29	Female	34	118	23.286113	Cloudy	3

2. انواع نمایش داده

دقت شود که تایپ اینها series هست:

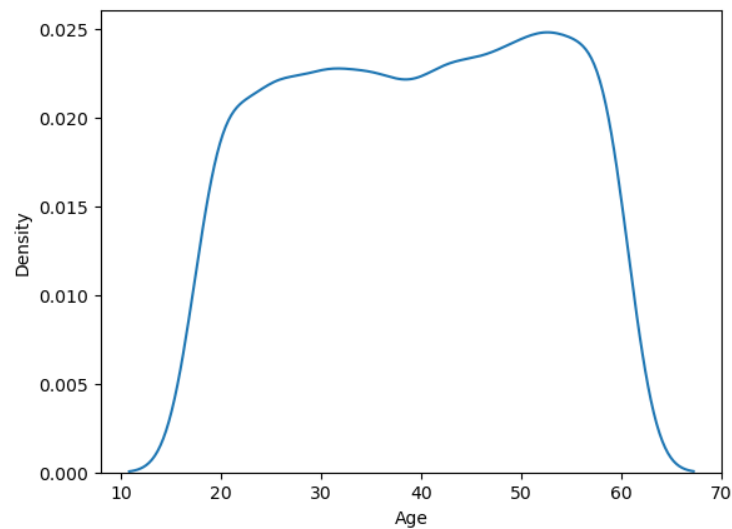
```
sns.kdeplot(df['BMI'])
plt.show()
```

نمودار KDE تک بعدی برای ستون BMI رسم می کند و توزیع چگالی این متغیر را نمایش می دهد.



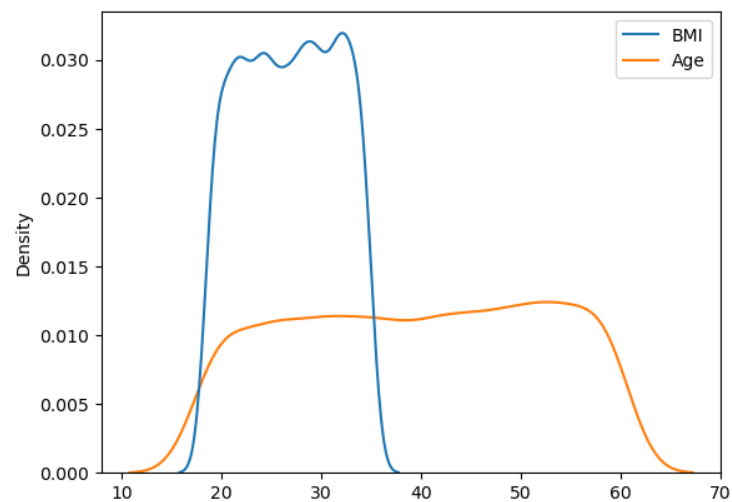
```
sns.kdeplot(df['Age'])  
plt.show()
```

نمودار KDE تک بعدی برای ستون Age رسم می کند و توزیع چگالی این متغیر را نشان می دهد.



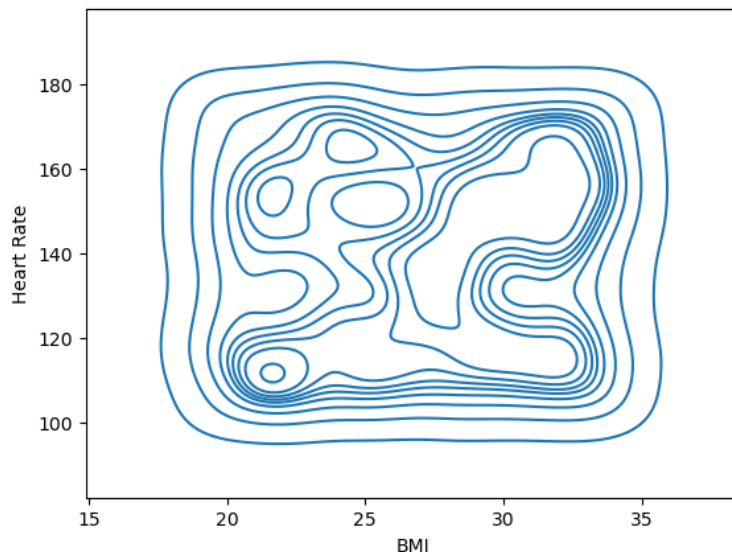
```
sns.kdeplot(df[['BMI', 'Age']])  
plt.show()
```

یک KDE چندمتغیره رسم می کند که توزیع چگالی هر دو متغیر BMI و Age را در یک نمودار مشترک نمایش می دهد.



```
sns.kdeplot(x=df['BMI'], y=df['Heart Rate'])
plt.show()
```

یک KDE دو بعدی برای متغیرهای BMI و Heart Rate رسم می‌کند که توزیع چگالی مشترک بین این دو متغیر را نشان می‌دهد.



نکته: دقت شود که اینها باید داده‌های عددی باشند.

3. رسم هیستوگرام

هیستوگرام‌ها ابزارهای تجسمی هستند که توزیع مجموعه‌ای از داده‌های پیوسته را نشان می‌دهند. در یک هیستوگرام، داده‌ها به مجموعه‌ای از بازه‌ها یا binها (معمولاً در محور X) تقسیم می‌شوند و برای هر کدام دارای تعدادی نقاط داده‌ای هستند که در هر bin، برابر با ارتفاع میله بالای آن bin است.

نمودار چگالی (همچنین به عنوان نمودار چگالی هسته شناخته می‌شود) ابزار تجسم دیگری برای ارزیابی توزیع داده‌ها است. می‌توان آن را به عنوان یک هیستوگرام هموار در نظر گرفت. پیک‌های نمودار چگالی به نمایش جایی که مقادیر در فاصله زمانی متمرکز هستند کمک می‌کند. حال به کمک Seaborn که یک کتابخانه تجسم داده بر اساس matplotlib در پایتون است، این را پیاده‌سازی می‌کنیم.

Syntax: `seaborn.histplot(data, x, y, hue, stat, bins, binwidth, discrete, kde, log_scale)`

Parameters:-

- **data:** input data in the form of Dataframe or Numpy array
- **x, y (optional):** key of the data to be positioned on the x and y axes respectively
- **hue (optional):** semantic data key which is mapped to determine the color of plot elements
- **stat (optional):** count, frequency, density or probability

Return: This method returns the matplotlib axes with the plot drawn on it.

استفاده از هیستوگرام در Seaborn و Matplotlib و کاربرد پارامتر bins

هیستوگرام نموداری است که توزیع داده‌های پیوسته را نشان می‌دهد و هر ستون نشان‌دهنده تعداد نقاط داده در هر دسته (بازه) است. برای استفاده از هیستوگرام، می‌توان از کتابخانه‌های Seaborn و Matplotlib استفاده کرد. پارامتر bins در اینجا برای تعیین تعداد بازه‌هایی که داده‌ها در آن قرار می‌گیرند استفاده می‌شود. این مقدار می‌تواند بر اساس نیاز تنظیم شود؛ تعداد بازه‌های بیشتر، نمودار دقیق‌تری ارائه می‌دهد، ولی اگر تعداد بازه‌ها بیش از حد زیاد باشد، ممکن است نویز بیشتری در نمودار دیده شود.

نکته: دقت شود که بهتر است تعداد bins در هر دو نمودار برابر باشد، مخصوصاً اگر قرار است دو توزیع به صورت مقایسه‌ای بررسی شوند. وقتی تعداد bins در هر دو نمودار یکسان باشد:

- مقایسه آسان‌تر می‌شود، چون هر ستون در هر دو نمودار یک بازه یکسان را نشان می‌دهد.
- وضوح و خوانایی افزایش می‌یابد، و می‌توان تفاوت‌ها و شباهت‌های توزیع داده‌ها را بهتر مشاهده کرد.

اگر تعداد bins در دو نمودار متفاوت باشد، بازه‌های دسته‌بندی برای هر نمودار متفاوت خواهد بود و این باعث می‌شود مقایسه دقیق توزیع‌ها سخت‌تر شود.

آیا محدودیتی مشاهده می‌کنید؟

بله، هیستوگرام‌ها ممکن است در مواردی که تعداد داده‌ها کم است یا توزیع داده‌ها ناهمبستگی است، دقت کافی نداشته باشند. همچنین، تنظیم تعداد مناسب bins می‌تواند چالش‌برانگیز باشد. استفاده از تعداد زیاد بازه‌ها ممکن است توزیع را ناپایدار و پرنوسان نشان دهد و در مقابل، تعداد کم بازه‌ها می‌تواند اطلاعات کمتری ارائه دهد.

پیاده سازی هیستوگرام با sns و matplotlib:

پیاده سازی کد با sns:

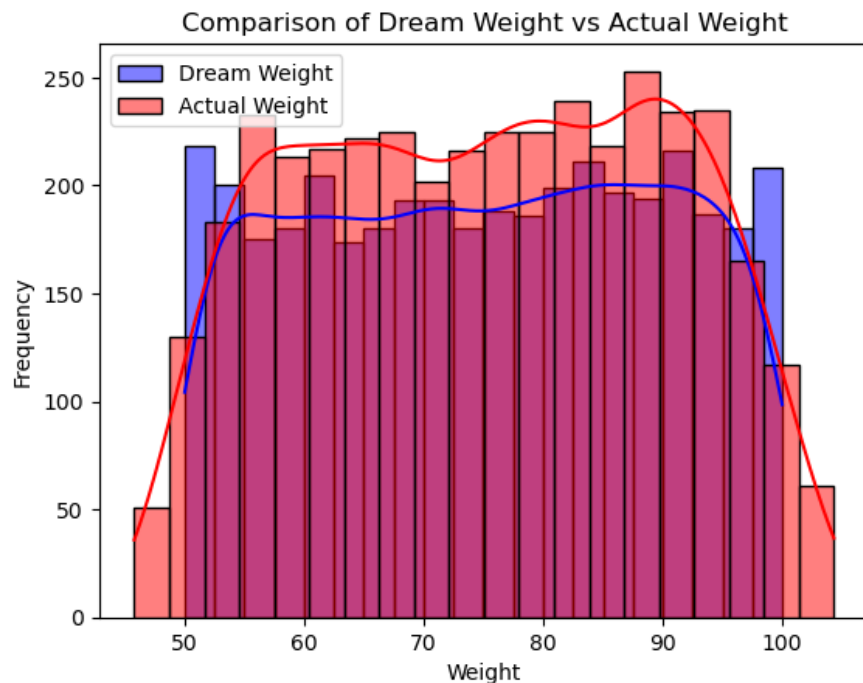
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.histplot(df['Dream Weight'], bins=20, color='blue', kde=True, label='Dream Weight')
sns.histplot(df['Actual Weight'], bins=20, color='orange', kde=True, label='Actual Weight')
```

```
plt.legend()
plt.xlabel("Weight")
plt.ylabel("Frequency")
plt.title("Comparison of Dream Weight vs Actual Weight")
plt.show()
```

ورودی‌ها و پارامترها:

- `df['Dream Weight']`: داده‌های ورودی که به صورت یک ستون از DataFrame `df` انتخاب شده است. این ستون مقادیر وزن ایده‌آل (Dream Weight) را شامل می‌شود.
- `bins=20`: تعداد بازه‌ها (یا ستون‌ها) در هیستوگرام را تعیین می‌کند. مقدار ۲۰ به این معنی است که داده‌ها به ۲۰ بخش تقسیم می‌شوند.
- `color='blue'`: رنگ ستون‌های هیستوگرام را به آبی تنظیم می‌کند.
- `kde=True`: تخمین چگالی هسته‌ای (KDE) را روی هیستوگرام رسم می‌کند که به صورت یک منحنی صاف روی هیستوگرام نمایش داده می‌شود و توزیع پیوسته داده‌ها را نشان می‌دهد.
- `label='Dream Weight'`: برچسبی برای این هیستوگرام تعریف می‌کند که در نمایش `legend` استفاده می‌شود.



پیاده سازی کد با matplotlib:

```
plt.hist(df['Dream Weight'], bins=20, color='blue', edgecolor='black', alpha=0.5, label='Dream Weight')

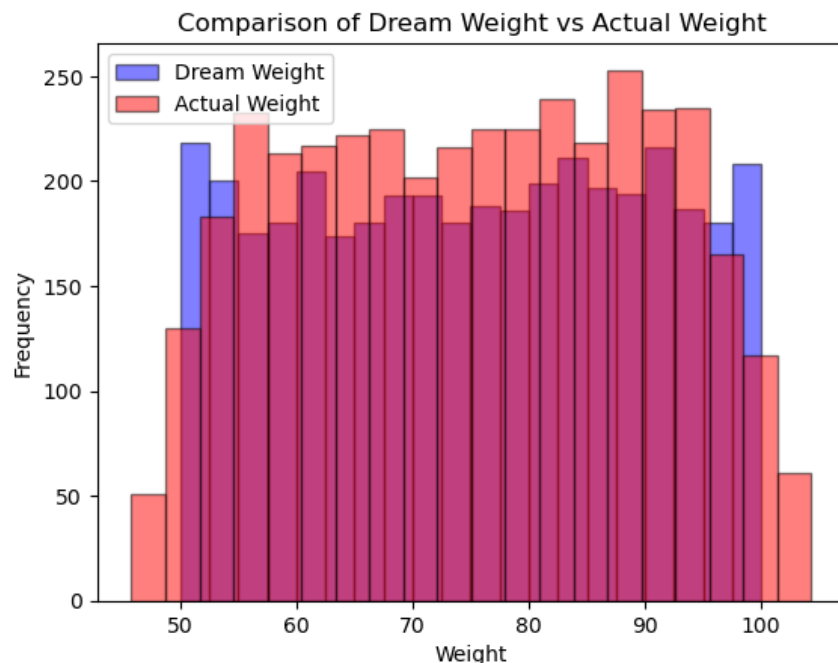
plt.hist(df['Actual Weight'], bins=20, color='red', edgecolor='black', alpha=0.5, label='Actual Weight')

plt.legend()
plt.xlabel("Weight")
plt.ylabel("Frequency")
plt.title("Comparison of Dream Weight vs Actual Weight")
plt.show()
```

توضیحات کد:

ورودی‌ها و پارامترها:

- **df['Dream Weight']**: داده‌های ورودی که به صورت یک ستون از DataFrame df انتخاب شده است. این ستون حاوی مقادیر وزن ایده‌آل (Dream Weight) است.
- **bins=20**: تعداد بازه‌ها (یا ستون‌ها) در هیستوگرام را مشخص می‌کند. مقدار ۲۰ به این معنی است که داده‌ها به ۲۰ بخش تقسیم می‌شوند.
- **color='blue'**: رنگ ستون‌های هیستوگرام را به آبی تنظیم می‌کند.
- **edgecolor='black'**: رنگ لبه‌های ستون‌ها را به مشکی تنظیم می‌کند، که به شفافیت و خوانایی بیشتر نمودار کمک می‌کند.
- **alpha=0.5**: میزان شفافیت رنگ را تنظیم می‌کند. مقدار 0.5 یعنی رنگ ستون‌ها نیمه‌شفاف است، که به همپوشانی دو نمودار کمک می‌کند تا هر دو به وضوح دیده شوند.
- **label='Dream Weight'**: برچسبی برای این هیستوگرام تعیین می‌کند که در legend نمایش داده می‌شود.



تفاوت استفاده از هیستوگرام:

- **Seaborn**: برای کارهای سریع، مصورسازی‌های زیبا، و تحلیل‌های آماری ساده‌تر، گزینه بهتری است.
- **Matplotlib**: انعطاف‌پذیری بالاتری دارد و برای نمودارهای پیچیده‌تر یا سفارشی‌سازی‌های خاص مناسب‌تر است.

4. استفاده از corr

همبستگی یک معیار آماری است که برای اندازه‌گیری و توصیف شدت و جهت رابطه بین دو متغیر استفاده می‌شود.

مفهوم همبستگی

همبستگی به ما نشان می‌دهد که چگونه دو متغیر با همدیگر تغییر می‌کنند. همبستگی می‌تواند مثبت، منفی یا صفر باشد:

- **مثبت**: وقتی یک متغیر افزایش می‌یابد، متغیر دیگر نیز افزایش می‌یابد.
- **منفی**: وقتی یک متغیر افزایش می‌یابد، متغیر دیگر کاهش می‌یابد.
- **صفر**: هیچ رابطه خطی مستقیمی بین دو متغیر وجود ندارد.

متد **corr** در **Pandas** برای محاسبه ماتریس همبستگی بین تمامی ستون‌های عددی در یک **DataFrame** استفاده می‌شود. این متد از پارامتر **method** برای تعیین نوع محاسبه همبستگی پشتیبانی می‌کند. سه مقدار اصلی برای این پارامتر، که انواع همبستگی هستند عبارتند از:

- **Pearson**: همبستگی پیرسون، که متداول‌ترین نوع همبستگی است.
- **Kendall**: همبستگی کندال، که بیشتر برای داده‌های رتبه‌بندی شده استفاده می‌شود.
- **Spearman**: همبستگی اسپیرمن، که برای داده‌های ترتیبی و غیرخطی مناسب است.

در این بخش از سه روش مختلف برای محاسبه ماتریس همبستگی استفاده شده است: **پیرسون**، **کندال**، و **اسپیرمن**.

❖ نوع اول - Pearson:

همبستگی پیرسون (**Pearson Correlation**): این نوع همبستگی شدت و جهت رابطه خطی بین دو متغیر پیوسته را اندازه‌گیری می‌کند و به صورت عددی بین -۱ و +۱ می‌باشد.

محدودیت: این روش تنها روابط خطی را اندازه‌گیری می‌کند و به تغییرات غیرخطی حساس نیست.

پیاده سازی و تحلیل نتایج:

```
# Pearson correlation: measures the linear relationship between two continuous variables
pearson_corr = df.corr(method='pearson')

# saving in a csv file
pearson_corr.to_csv('Q1_pearson_correlation_matrix.csv', index=True)

print("Pearson Correlation Matrix:\n", pearson_corr)
```

✓ 0.0s

Pearson Correlation Matrix:

	ID	Calories Burn	Dream Weight	Actual Weight	\
ID	1.000000	-0.017108	-0.001532	-0.001108	
Calories Burn	-0.017108	1.000000	0.012749	0.010444	
Dream Weight	-0.001532	0.012749	1.000000	0.981377	
Actual Weight	-0.001108	0.010444	0.981377	1.000000	
Age	0.009428	-0.001113	0.006689	0.003368	
Duration	0.032130	0.021793	0.016116	0.020490	
Heart Rate	0.007062	-0.035889	-0.001855	-0.002543	
BMI	0.003625	0.012602	0.002237	-0.003703	
Exercise Intensity	-0.004103	0.010819	0.000087	-0.002670	

	Age	Duration	Heart Rate	BMI	\
ID	0.009428	0.032130	0.007062	0.003625	
Calories Burn	-0.001113	0.021793	-0.035889	0.012602	
Dream Weight	0.006689	0.016116	-0.001855	0.002237	
Actual Weight	0.003368	0.020490	-0.002543	-0.003703	
Age	1.000000	-0.006567	-0.008939	-0.002915	
Duration	-0.006567	1.000000	0.014941	0.020203	
Heart Rate	-0.008939	0.014941	1.000000	0.015555	
BMI	-0.002915	0.020203	0.015555	1.000000	
Exercise Intensity	0.005565	0.023195	-0.004510	-0.020881	

	Exercise Intensity
ID	-0.004103
...	
Duration	0.023195
Heart Rate	-0.004510
BMI	-0.020881
Exercise Intensity	1.000000

در این ماتریس همبستگی، رابطه بین هر جفت از ستون‌های عددی DataFrame با استفاده از همبستگی پیرسون محاسبه شده است.

- خودهمبستگی (مقدار ۱ در قطر اصلی):

○ هر ستون با خودش همبستگی کامل (1+) دارد. این مقادیر در قطر اصلی ماتریس قرار دارند.

- همبستگی بالای ستون‌های Dream Weight و Actual Weight:

○ مقدار همبستگی بین Dream Weight و Actual Weight برابر با 0.981377 است، که نشان‌دهنده یک همبستگی بسیار قوی و مثبت بین این دو ستون است. این نشان می‌دهد که افرادی که وزن ایده‌آل بالاتری دارند، به طور کلی وزن واقعی بیشتری نیز دارند.

- همبستگی ضعیف بین Actual Weight و Calories Burn:

- مقدار همبستگی بین Actual Weight و Calories Burn برابر با 0.010444 است، که بسیار نزدیک به صفر است. این مقدار نشان می‌دهد که رابطه خطی معناداری بین این دو متغیر وجود ندارد و تغییرات در Calories Burn تأثیر مشخصی بر Actual Weight ندارد.

- همبستگی بسیار پایین بین بیشتر ستون‌ها:

- بسیاری از جفت‌های دیگر مقادیر همبستگی نزدیک به صفر دارند (مثلاً Age و Heart Rate با مقدار -0.001855) این نشان می‌دهد که بیشتر متغیرها رابطه خطی قوی‌ای با هم ندارند.

- تفسیر همبستگی‌های منفی کوچک:

- بعضی از مقادیر همبستگی، مثل رابطه بین Actual Weight و Exercise Intensity، منفی (-0.002670) است اما مقدار آن بسیار کوچک است. این مقادیر نشان می‌دهند که رابطه‌ی ضعیف و منفی بین این متغیرها وجود دارد، اما این رابطه بسیار ناچیز است و تقریباً بی‌معنی است.

نتیجه‌گیری کلی

این ماتریس نشان می‌دهد که رابطه‌های خطی قوی و معناداری بین بیشتر ستون‌ها وجود ندارد، به جز بین Dream Weight و Actual Weight که همبستگی بسیار بالایی دارند.

همچنین نتایج رو داخل یک CSV ذخیره کردیم.

❖ نوع دوم - Kendall:

همبستگی کندال (Kendall Correlation): این نوع همبستگی شدت و جهت رابطه رتبه‌ای بین دو متغیر را اندازه‌گیری می‌کند. در این روش، به جای مقادیر اصلی، رتبه‌ها مقایسه می‌شوند.

این روش مناسب برای داده‌هایی هستند که دارای ترتیب (رتبه) هستند، یا وقتی داده‌ها به‌شدت نرمال نیستند.

```
# Kendall correlation: measures the ordinal association between two variables
kendall_corr = df.corr(method='kendall')

# saving in a csv file
kendall_corr.to_csv('Q1_kendall_correlation_matrix.csv', index=True)

print("\nKendall Correlation Matrix:\n", kendall_corr)
```

✓ 0.0s

Kendall Correlation Matrix:

	ID	Calories Burn	Dream Weight	Actual Weight	\
ID	1.000000	-0.011487	-0.001133	-0.001143	
Calories Burn	-0.011487	1.000000	0.007947	0.006680	
Dream Weight	-0.001133	0.007947	1.000000	0.875449	
Actual Weight	-0.001143	0.006680	0.875449	1.000000	
Age	0.005884	-0.000145	0.004405	0.002358	
Duration	0.021634	0.014629	0.010710	0.013658	
Heart Rate	0.004484	-0.023931	-0.000659	-0.001105	
BMI	0.002055	0.008455	0.001327	-0.002156	
Exercise Intensity	-0.002696	0.007602	-0.000115	-0.001949	

	Age	Duration	Heart Rate	BMI	\
ID	0.005884	0.021634	0.004484	0.002055	
Calories Burn	-0.000145	0.014629	-0.023931	0.008455	
Dream Weight	0.004405	0.010710	-0.000659	0.001327	
Actual Weight	0.002358	0.013658	-0.001105	-0.002156	
Age	1.000000	-0.004234	-0.005997	-0.001844	
Duration	-0.004234	1.000000	0.010296	0.013859	
Heart Rate	-0.005997	0.010296	1.000000	0.010663	
BMI	-0.001844	0.013859	0.010663	1.000000	
Exercise Intensity	0.004623	0.016639	-0.003300	-0.014774	

	Exercise Intensity
...	
Duration	0.016639
Heart Rate	-0.003300
BMI	-0.014774
Exercise Intensity	1.000000

در این تصویر، ماتریس همبستگی کندال محاسبه و نمایش داده شده است. این ماتریس نشان‌دهنده همبستگی رتبه‌ای بین ستون‌های عددی موجود در DataFrame است. روش کندال به‌ویژه برای داده‌های ترتیبی یا داده‌هایی که دارای توزیع غیرنرمال هستند مناسب است، زیرا به مقایسه رتبه‌ها می‌پردازد و به همین دلیل نسبت به روش پیرسون به‌نویز کمتر حساس است.

- خودهمبستگی (مقدار ۱ در قطر اصلی):

○ هر ستون با خودش همبستگی کامل دارد که به صورت 1.000000 در قطر اصلی ماتریس نشان داده شده است. این مقدار طبیعی است زیرا هر متغیر با خودش رابطه کامل دارد.

- همبستگی بالا بین Actual Weight و Dream Weight:

- مقدار همبستگی کندال بین Actual Weight و Dream Weight برابر با 0.875449 است. این مقدار نشان می‌دهد که رابطه‌ی رتبه‌ای قوی و مثبت بین وزن ایده‌آل و وزن واقعی وجود دارد. به این معنی که افرادی که وزن ایده‌آل بالاتری دارند، به طور کلی رتبه‌ی بالاتری نیز در وزن واقعی دارند.

- همبستگی پایین بین Actual Weight و Calories Burn:

- مقدار همبستگی بین Actual Weight و Calories Burn برابر با -0.000659 است، که بسیار نزدیک به صفر است و نشان‌دهنده‌ی عدم وجود رابطه‌ی رتبه‌ای بین این دو متغیر است. به این معنی که تغییرات در Calories Burn تأثیر مشخصی بر رتبه‌ی Actual Weight ندارد.

- همبستگی بسیار پایین یا تقریباً صفر بین بیشتر ستون‌ها:

- بسیاری از جفت‌های دیگر، مقادیر همبستگی بسیار پایینی دارند، مانند BMI و Duration با مقدار 0.013659. که نشان‌دهنده‌ی نبود رابطه‌ی رتبه‌ای معنادار بین آن‌ها است. این نتیجه نشان می‌دهد که اکثر ستون‌ها در این DataFrame همبستگی رتبه‌ای قوی‌ای با هم ندارند.

- همبستگی منفی ضعیف بین Exercise Intensity و برخی ستون‌ها:

- چند مقدار همبستگی منفی ضعیف، مثل رابطه بین Exercise Intensity و Heart Rate با مقدار -0.003300، وجود دارد. این مقادیر نشان می‌دهند که رابطه‌ی رتبه‌ای ضعیف و منفی بین این متغیرها وجود دارد، ولی این رابطه بسیار کم است و تأثیر معناداری ندارد.

نتیجه‌گیری کلی

این ماتریس همبستگی کندال نشان می‌دهد که بیشتر متغیرها در این DataFrame رابطه رتبه‌ای قوی‌ای با یکدیگر ندارند، به‌جز Actual Weight و Dream Weight که همبستگی بالایی با هم دارند. این نشان‌دهنده این است که بیشتر متغیرها مستقل از یکدیگر هستند و هیچ‌کدام تأثیر معناداری بر دیگری ندارند.

همچنین نتایج رو داخل یک CSV ذخیره کردیم.

❖ نوع سوم - Spearman

همبستگی اسپیرمن (Spearman Correlation): این روش نیز مانند کندال برای داده‌های رتبه‌ای استفاده می‌شود و به ویژه برای داده‌هایی که روابط غیرخطی دارند مناسب است.

در این روش داده‌ها به رتبه تبدیل شده و سپس از فرمول پیرسون برای محاسبه همبستگی بین رتبه‌ها استفاده می‌شود.

پیاده سازی و تحلیل نتایج:

```
# Spearman correlation: assesses the monotonic relationship between two variables
spearman_corr = df.corr(method='spearman')

# saving in a csv file
spearman_corr.to_csv('Q1_spearman_correlation_matrix.csv', index=True)

print("\nSpearman Correlation Matrix:\n", spearman_corr)
```

✓ 0.0s

Spearman Correlation Matrix:

	ID	Calories Burn	Dream Weight	Actual Weight	\
ID	1.000000	-0.017209	-0.001781	-0.001568	
Calories Burn	-0.017209	1.000000	0.012212	0.010197	
Dream Weight	-0.001781	0.012212	1.000000	0.982181	
Actual Weight	-0.001568	0.010197	0.982181	1.000000	
Age	0.008983	-0.000442	0.006468	0.003453	
Duration	0.032274	0.021783	0.015873	0.020332	
Heart Rate	0.006990	-0.035599	-0.001032	-0.001812	
BMI	0.003146	0.012587	0.001970	-0.003168	
Exercise Intensity	-0.004004	0.010869	0.000099	-0.002620	

	Age	Duration	Heart Rate	BMI	\
ID	0.008983	0.032274	0.006990	0.003146	
Calories Burn	-0.000442	0.021783	-0.035599	0.012587	
Dream Weight	0.006468	0.015873	-0.001032	0.001970	
Actual Weight	0.003453	0.020332	-0.001812	-0.003168	
Age	1.000000	-0.006229	-0.008587	-0.002861	
Duration	-0.006229	1.000000	0.015226	0.020352	
Heart Rate	-0.008587	0.015226	1.000000	0.015756	
BMI	-0.002861	0.020352	0.015756	1.000000	
Exercise Intensity	0.006412	0.023383	-0.004833	-0.021039	

	Exercise Intensity
...	
Duration	0.023383
Heart Rate	-0.004833
BMI	-0.021039
Exercise Intensity	1.000000

در این تصویر، ماتریس همبستگی اسپیرمن محاسبه و نمایش داده شده است. روش اسپیرمن، همبستگی رتبه‌ای بین متغیرها را محاسبه می‌کند و برای داده‌هایی که دارای روابط غیرخطی هستند یا داده‌های ترتیبی مناسب است. این روش، همبستگی مونوتونیک بین متغیرها را اندازه‌گیری می‌کند، به این معنا که اگر یکی از متغیرها افزایش یا کاهش یابد، دیگری هم به‌طور پیوسته در یک جهت تغییر می‌کند (هرچند این تغییر لزوماً خطی نیست).

- خودهمبستگی (مقدار ۱ در قطر اصلی):

○ هر ستون با خودش همبستگی کامل دارد که به صورت 1.000000 در قطر اصلی ماتریس نشان داده شده است. این مقدار طبیعی است، زیرا هر متغیر با خودش رابطه کامل دارد.

- همبستگی بالا بین **Dream Weight** و **Actual Weight**:

○ مقدار همبستگی اسپیرمن بین **Dream Weight** و **Actual Weight** برابر با 0.982181 است. این مقدار نشان می‌دهد که رابطه‌ی رتبه‌ای بسیار قوی و مثبت بین این دو ستون وجود دارد. یعنی اگر رتبه‌ی **Dream Weight** بالا باشد، رتبه‌ی **Actual Weight** نیز بالا خواهد بود و برعکس.

- همبستگی پایین یا نزدیک به صفر بین اکثر متغیرها:

○ بیشتر جفت‌های متغیرها مقادیر همبستگی نزدیک به صفر دارند، مانند **Actual Weight** و **Calories Burn** با مقدار 0.010197 این مقدار نشان می‌دهد که رابطه‌ی مونوتونیک معناداری بین این دو متغیر وجود ندارد و تغییرات در یک متغیر تأثیر مشخصی بر دیگری ندارد.

- همبستگی ضعیف بین **Exercise Intensity** و دیگر متغیرها:

○ در این ماتریس نیز، **Exercise Intensity** با سایر متغیرها همبستگی‌های بسیار ضعیفی دارد (مانند -0.004833 با **Heart Rate**) این نشان می‌دهد که تغییرات رتبه‌ای در شدت تمرین ورزشی تأثیر خاصی بر سایر متغیرها ندارد.

- مقایسه با ماتریس‌های پیرسون و کندال:

○ مشابه ماتریس کندال، همبستگی اسپیرمن نیز نشان می‌دهد که اکثر متغیرها همبستگی مونوتونیک معناداری با هم ندارند، به جز **Dream Weight** و **Actual Weight** که دارای همبستگی قوی و مثبت هستند.

نتیجه‌گیری کلی

این ماتریس اسپیرمن نشان می‌دهد که بسیاری از متغیرها در این **DataFrame** رابطه مونوتونیک قوی‌ای با یکدیگر ندارند. تنها استثنا، جفت متغیرهای **Dream Weight** و **Actual Weight** است که همبستگی قوی و مثبت دارند. این نشان می‌دهد که در رتبه‌بندی این دو متغیر، هرچه وزن ایده‌آل فرد بالاتر باشد، وزن واقعی او نیز به همان نسبت بالاتر است.

5. حذف پارامترهای غیر عددی

- پارامتر `axis`: مشخص می‌کند که آیا می‌خواهید سطرها (`axis=0`) یا ستون‌ها (`axis=1`) را حذف کنید.
- پارامتر `inplace`: اگر مقدار این پارامتر `True` باشد، تغییرات مستقیماً روی `DataFrame` اعمال می‌شوند و `DataFrame` جدیدی برنگردانده می‌شود. اگر `False` باشد (پیش‌فرض)، `DataFrame` جدیدی با تغییرات برمی‌گردد و `DataFrame` اصلی بدون تغییر باقی می‌ماند.

کد:

فرض می‌کنیم که می‌خواهیم چندین ستون از `DataFrame` را حذف کنیم:

```
df.drop(['Gender', 'Exercise'], axis=1, inplace=True)
```

در این کد، ستون‌های `Gender` و `Exercise` از `DataFrame` حذف می‌شوند و چون `inplace=True` است، تغییرات روی `DataFrame` اصلی اعمال می‌شوند.

۲. سوال شماره ۲:

بر اساس توضیحات داده شده در نوت بوک و دیتای snapp.csv به مسئله مطرح شده پاسخ دهید. مسئله از نوع anomaly detection است که با استفاده از Z-score حل می‌شود.

این سوال عملی می‌باشد.

➤ تعریف

تشخیص ناهنجاری (Anomaly Detection):

تشخیص ناهنجاری یک روش در علم داده و یادگیری ماشین است که به دنبال شناسایی نقاط داده‌ای است که با سایر داده‌ها تفاوت قابل توجهی دارند. این ناهنجاری‌ها معمولاً نشان‌دهنده رفتارهای غیرمعمول یا خطاها هستند و در بسیاری از موارد، کشف آن‌ها می‌تواند اطلاعات مهمی در مورد الگوهای غیرعادی فراهم کند.

Z-Score (متیاز استاندارد):

Z-score یک معیار آماری است که مشخص می‌کند یک مقدار خاص چقدر با میانگین مجموعه داده فاصله دارد. فرمول محاسبه Z-score به شکل زیر است:

$$z = \frac{X - \mu}{\sigma}$$

که در آن:

- X = مقدار داده
- μ = میانگین داده‌ها
- Σ = انحراف معیار داده‌ها

مقدار Z-score مثبت نشان‌دهنده مقداری بالاتر از میانگین و مقدار منفی نشان‌دهنده مقداری پایین‌تر از میانگین است. اگر مقدار Z-score بیش از حد (با یک threshold خاص) از میانگین فاصله داشته باشد، آن نقطه به عنوان یک ناهنجاری در نظر گرفته می‌شود.

➤ ساختار کد:

- بارگذاری و بررسی اولیه داده‌ها

در اولین مرحله، داده‌ها از فایل `snapp.csv` بارگذاری کرده و چند سطر ابتدایی آن‌ها را نمایش می‌دهیم. این کار کمک می‌کند تا ساختار داده‌ها و نوع اطلاعات موجود را بهتر درک کنیم.

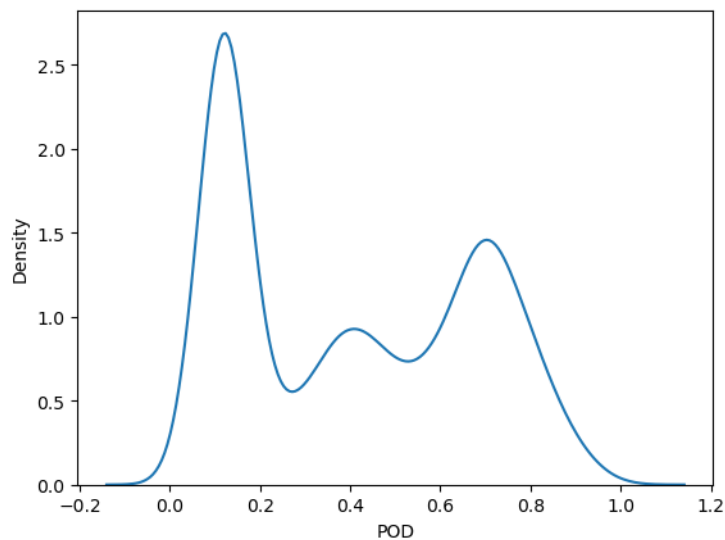
```
import pandas as pd
df = pd.read_csv("snapp.csv")
print(df.head())
```

	POD	AvgSpentOnSnappFood
0	0.095239	2.931748e+06
1	0.105493	3.517870e+06
2	0.123370	3.481772e+06
3	0.141007	3.531373e+06
4	0.102860	3.440841e+06

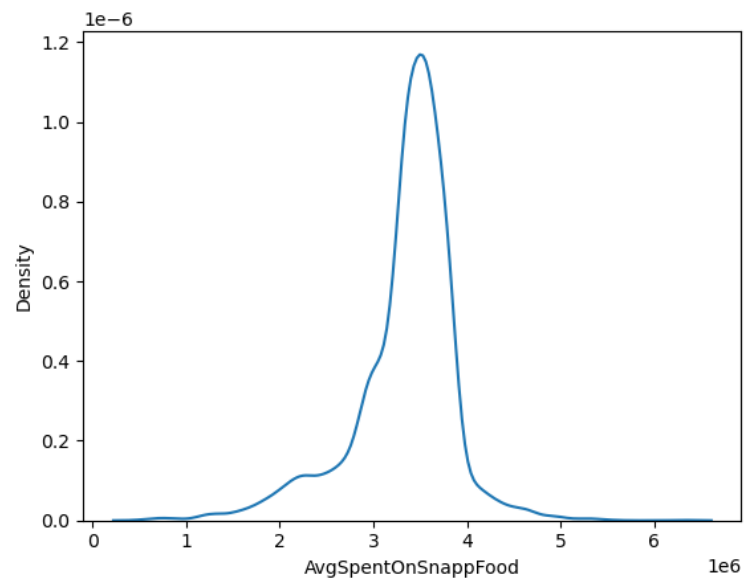
- نمایش داده‌های اولیه

در این بخش نمودار داده‌ها و نسبتان با هم را، جهت آشنایی بیشتر با داده‌ها رسم می‌کنیم.

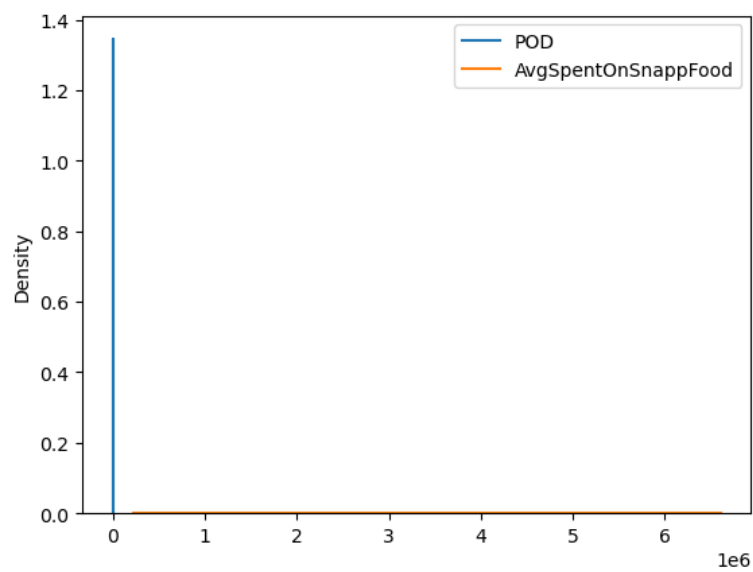
```
sns.kdeplot(df['POD'])
plt.show()
```



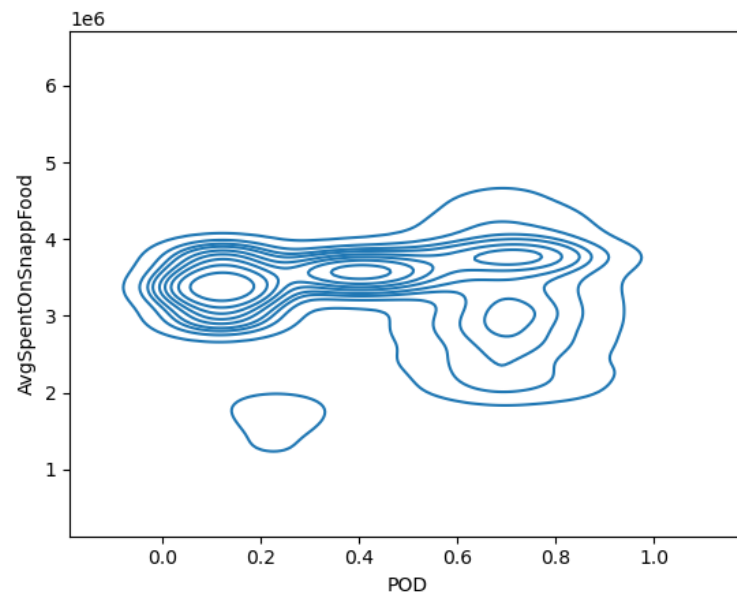
```
sns.kdeplot(df['AvgSpentOnSnappFood'])  
plt.show()
```



```
sns.kdeplot(df[['POD', 'AvgSpentOnSnappFood']])  
plt.show()
```



```
sns.kdeplot(x=df['POD'], y=df['AvgSpentOnSnappFood'])
plt.show()
```



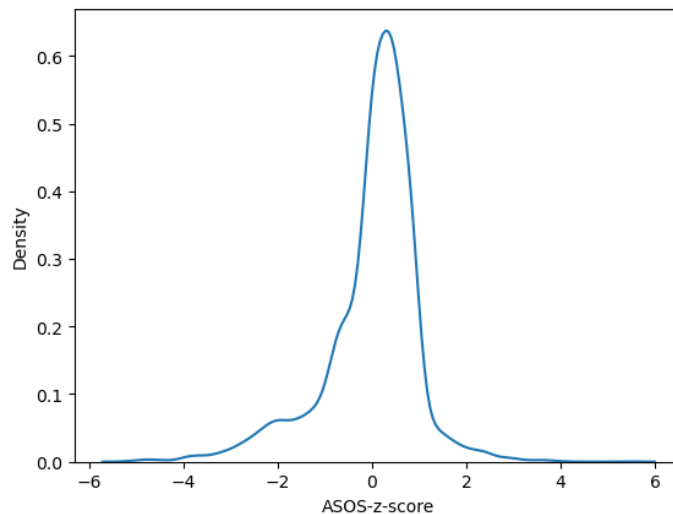
• محاسبه z-score

در این مرحله، z-score برای ستون AvgSpentOnSnappFood محاسبه می‌شود تا نقاطی که به طور قابل توجهی از میانگین فاصله دارند شناسایی شوند و در ادامه آن را نمایش می‌دهیم.

```
df["ASOS-z-score"] = df[['AvgSpentOnSnappFood']].apply(stats.zscore)
df.head()
```

	POD	AvgSpentOnSnappFood	ASOS-z-score
0	0.095239	2.931748e+06	-0.749173
1	0.105493	3.517870e+06	0.325485
2	0.123370	3.481772e+06	0.259298
3	0.141007	3.531373e+06	0.350242
4	0.102860	3.440841e+06	0.184252

```
sns.kdeplot(df['ASOS-z-score'])
plt.show()
```



• شناسایی نقاط ناهنجار

پس از محاسبه Z-score، نقاطی که Z-score آن‌ها بیش از 1 یا کمتر از -1 است، به عنوان ناهنجاری در نظر گرفته شده و فیلتر می‌شوند. این مقادیر به طور قابل توجهی از میانگین فاصله دارند. سپس تعداد آنها را محاسبه می‌کنیم.

```
ASOS_anomalies = pd.DataFrame(abs(df['ASOS-z-score'])>=1)
display(ASOS_anomalies)
```

```
ASOS_true_count = ASOS_anomalies.values.sum()
print("Total True values in DataFrame:", ASOS_true_count)
```

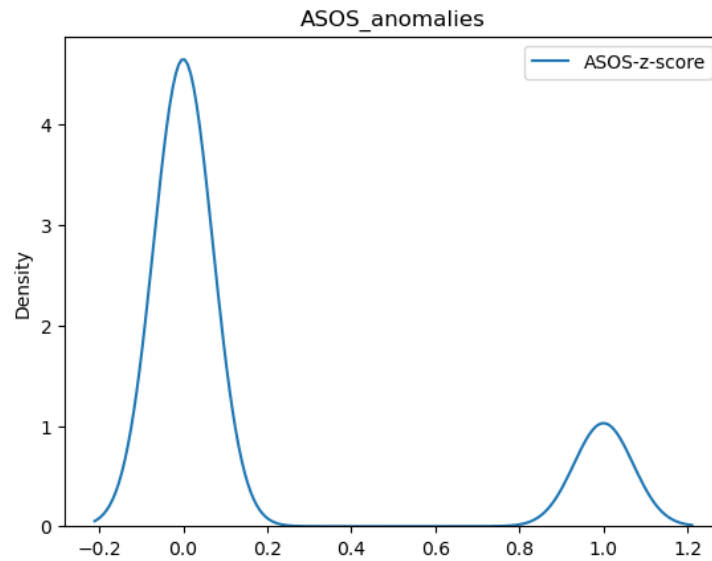
Total True values in DataFrame: 894

ASOS-z-score	
0	False
1	False
2	False
3	False
4	False
...	...
4938	True
4939	True
4940	False
4941	True
4942	False

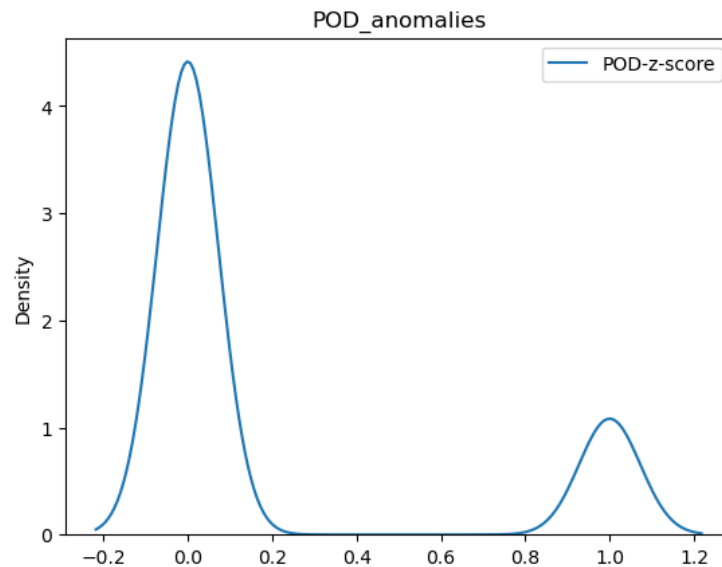
- رسم نمودارهای توزیع

برای مقایسه توزیع اصلی داده‌ها و توزیع z-score، از نمودار KDE استفاده شده است. این نمودارها کمک می‌کنند تا ببینیم چگونه توزیع اصلی داده‌ها پس از تبدیل به z-score تغییر کرده است.

```
sns.kdeplot(ASOS_anomalies)
plt.title('ASOS_anomalies')
plt.show()
```



در ادامه همین مراحل را برای ستون POD هم انجام می‌دهیم. اما در اینجا دیگر تنها به نشان دادن نتیجه آن اکتفا می‌کنیم.



نکته: برای فیلتر کردن POD با z-score ابتدا 1 را زدیم تعداد داده‌های بسیار زیاد شد، سپس 1.5 را امتحان کردیم بسیار کم شد، بنابراین از 1.2 استفاده کردیم.

نکات مهم

- **پراکندگی داده‌ها:** توزیع داده‌ها و مقدار انحراف معیار آن‌ها باید در نظر گرفته شود. استفاده از Z-SCORE زمانی مناسب است که داده‌ها توزیع نرمال داشته باشند.
- **آستانه انتخابی:** انتخاب آستانه برای شناسایی ناهنجاری‌ها باید با دقت صورت گیرد. این مقدار بر اساس نیاز و ساختار داده می‌تواند تغییر کند.
- **بررسی بیشتر نقاط ناهنجار:** نقاطی که به عنوان ناهنجاری شناسایی شده‌اند، باید به دقت بررسی شوند تا مشخص شود که آیا این مقادیر واقعاً غیرعادی‌اند یا فقط به دلیل واریانس طبیعی در داده‌ها ایجاد شده‌اند.

تحلیل و مقایسه نتایج

در اینجا، دو نمودار چگالی (Density Plot) از z-score های مربوط به ویژگی‌های POD و ASOS نمایش داده شده است که نشان‌دهنده‌ی توزیع داده‌های استاندارد شده‌ی این ویژگی‌ها است. این نمودارها پس از اعمال فرآیند تشخیص ناهنجاری و تبدیل مقادیر به Z-SCORE، به این شکل خاص درآمده‌اند.

1. توزیع z-score برای POD و ASOS

- در هر دو نمودار، ما یک پیک اصلی (در نزدیکی صفر) و یک پیک کوچک‌تر در محدوده‌ی مثبت (نزدیک به ۱ یا بالاتر) مشاهده می‌کنیم.
- این نوع توزیع نشان‌دهنده وجود دو مجموعه داده است که یکی در مرکز توزیع (پیک اصلی) و دیگری در سمت راست پیک اصلی قرار دارد (پیک کوچک‌تر). پیک اصلی نزدیک به صفر است که نشان می‌دهد اکثر داده‌ها از میانگین فاصله زیادی ندارند و در یک محدوده معمول و طبیعی قرار دارند.

2. پیک دوم در محدوده مثبت:

- پیک دوم که به مقدار ۱ نزدیک است، نشان می‌دهد که گروهی از داده‌ها به طور واضح با بقیه‌ی داده‌ها متفاوت‌اند. این داده‌ها به عنوان نقاطی شناسایی می‌شوند که از میانگین فاصله بیشتری دارند و به عنوان **ناهنجاری‌ها** یا **outliers** در نظر گرفته می‌شوند.
- این پیک دوم ممکن است نشان‌دهنده‌ی دسته‌ای از داده‌ها باشد که به دلیل ویژگی‌های خاص خود از الگوی معمول داده‌ها پیروی نمی‌کنند. برای مثال، در داده‌های مربوط به SnappFood، این مقادیر می‌تواند به کاربران خاصی اشاره داشته باشد که مصرف بیشتری نسبت به بقیه داشته‌اند.

3. شکل کلی توزیع:

- شکل کلی توزیع هر دو ویژگی POD و ASOS مشابه است و این نشان می‌دهد که هر دو مجموعه داده دارای گروه‌های مشابهی از نقاط ناهنجار هستند. این شباهت ممکن است نشان دهد که الگوی ناهنجاری در این دو ویژگی مشابه است، یا این دو ویژگی با یکدیگر همبستگی دارند.

چرا نمودارها به این شکل درآمده‌اند؟

این نوع توزیع با دو پیک می‌تواند به دلایل زیر ایجاد شده باشد:

1. تقسیم‌بندی طبیعی در داده‌ها:

- داده‌ها به طور طبیعی به دو گروه تقسیم شده‌اند. ممکن است که برخی کاربران دارای الگوی مصرف خاصی باشند که به شدت با سایر کاربران متفاوت است.

2. ناهنجاری‌های واقعی:

- پیک دوم می‌تواند نشان‌دهنده‌ی نقاط ناهنجار باشد که به عنوان انحرافات مثبت از میانگین شناسایی شده‌اند. این نقاط در Z-SCORE تبدیل شده‌اند و به دلیل فاصله زیاد از میانگین، در سمت راست نمودار قرار گرفته‌اند.

3. استانداردسازی داده‌ها:

- با تبدیل داده‌ها به Z-SCORE، توزیع اصلی داده‌ها به شکلی استاندارد درآمده است که میانگین آن صفر است و انحراف معیار یک است، این تبدیل به ما کمک می‌کند تا نقاطی که به طور معناداری با بقیه داده‌ها تفاوت دارند را شناسایی کنیم.

نتیجه‌گیری

در این تحلیل، دو پیک مجزا در توزیع Z-score های ویژگی‌های POD و ASOS شناسایی شد. این پیک‌ها نشان‌دهنده‌ی وجود گروهی از داده‌ها هستند که به عنوان ناهنجاری شناسایی شده‌اند. این ناهنجاری‌ها ممکن است به دلیل رفتارهای خاص کاربران یا ویژگی‌های خاص داده‌ها ایجاد شده باشند. با توجه به نتایج، پیشنهاد می‌شود که این نقاط ناهنجار به طور دقیق‌تر بررسی شوند و از روش‌های دیگر برای تأیید نتایج استفاده شود.

پیشنهادهای

1. **بررسی دقیق‌تر ناهنجاری‌ها:** داده‌های شناسایی شده به عنوان ناهنجاری باید به صورت جداگانه بررسی شوند تا دلایل احتمالی آن‌ها مشخص شود. این بررسی می‌تواند شامل تحلیل رفتار کاربران با هزینه‌های بالا باشد.
2. **استفاده از روش‌های دیگر تشخیص ناهنجاری:** در صورتی که داده‌ها دارای توزیع غیرنرمال باشند یا ساختار پیچیده‌ای داشته باشند، روش Z-score ممکن است بهترین گزینه نباشد. روش‌هایی مانند **IQR (Interquartile Range)** یا **DBSCAN** برای شناسایی ناهنجاری‌های غیرفوری‌ای می‌توانند مفید باشند.
3. **تحلیل‌های بیشتری روی کاربران پرهزینه:** کاربرانی که هزینه‌های بالایی داشته‌اند، می‌توانند به عنوان یک گروه خاص مورد تحلیل قرار گیرند. با دسته‌بندی کاربران بر اساس هزینه، می‌توان تحلیل‌های بیشتری برای بهبود خدمات و شناسایی نیازهای آن‌ها انجام داد.

۳. سوال شماره ۳:

طراحی سیستم پیشنهاد کتاب با استفاده از Collaborative Filtering

به عنوان یک داده‌کاو در یک فروشگاه آنلاین کتاب استخدام شده‌اید و از شما خواسته شده است تا یک سیستم پیشنهاد کتاب طراحی کنید. برای این کار، از دیتاست زیر استفاده کنید:

دیتاست: یک فایل CSV به نام `book_ratings.csv` با اطلاعات زیر:

`user_id`: شناسه‌ی کاربر

`book_id`: شناسه‌ی کتاب

`Rating`: امتیاز کاربر به کتاب (از 1 تا 5)

وظایف:

- بارگذاری و آماده‌سازی داده‌ها: با استفاده از کتابخانه‌های `numpy` و `pandas` داده‌ها را بارگذاری و یک ماتریس امتیازدهی بسازید.
- تحلیل داده‌ها: یک تحلیل ابتدایی از داده‌ها انجام دهید (تعداد کاربران، تعداد کتاب‌ها، توزیع امتیازها و غیره).

پیاده‌سازی Collaborative Filtering با روش User-Based:

یک تابع بنویسید که شباهت بین کاربران را محاسبه کند (با استفاده از `cosine similarity` یا `Pearson correlation`).

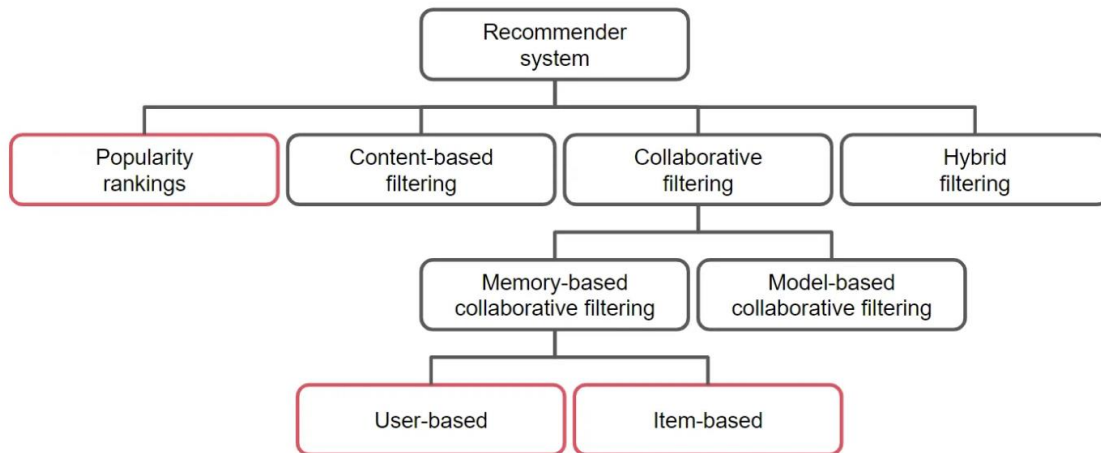
با استفاده از شباهت‌ها، برای یک کاربر خاص، ۵ کتاب با بالاترین پیش‌بینی امتیاز را پیشنهاد دهید.

گزارش: نتایج و تحلیل‌های خود را در یک نوت‌بوک مستند کنید، توضیح دهید که کدام روش بهتر عمل کرده و دلایل آن چیست.

این سوال عملی می‌باشد.

➤ تعریف:

یک سیستم پیشنهاد دهنده (Recommandation system) با دسته بندی زیر شناخته می شود.



ما در این سوال با روش Collaborative Filtering با متد User-Based کار می کنیم.

تعریف User-Based Collaborative Filtering

در این روش، هدف محاسبه شباهت بین کاربران بر اساس رفتارهای مشابهشان در نمره دهی به آیتمها است. این روش بر اساس فرضیه "کاربران مشابه، سلیقه های مشابهی دارند" کار می کند و از داده های رفتاری گذشته برای پیش بینی نمرات و پیشنهاد آیتم های جدید بهره می برد.

مراحل کلی پیاده سازی User-Based Collaborative Filtering

1. ساخت ماتریس امتیازدهی کاربران-آیتمها: ثبت نمرات کاربران برای آیتمها در یک ماتریس.
2. پیدا کردن نزدیک ترین همسایه ها برای هر کاربر: شناسایی کاربرانی با بیشترین شباهت.
3. محاسبه شباهت بین کاربران: محاسبه شباهت ها با استفاده از معیارهایی مانند Cosine Similarity و Pearson.
4. پیش بینی نمرات و پیشنهاد آیتمها: استفاده از نمرات همسایه ها برای ارائه پیشنهادها.

مزایا و معایب User-Based Collaborative Filtering

- مزایا:
 - کارایی در محیط های پویا بدون نیاز به اطلاعات محتوای آیتمها.
 - شخصی سازی بالا به دلیل تمرکز بر رفتار واقعی کاربران.
- معایب:
 - مقیاس پذیری: افزایش هزینه محاسباتی با افزایش کاربران و آیتمها.
 - Cold Start Problem: عملکرد ضعیف برای کاربران جدید.
 - Sparsity Issue: مشکل در پیدا کردن شباهت در ماتریس های پراکنده.

K-Nearest Neighbors:

استفاده از **K-Nearest Neighbors** یا **(KNN)**: استفاده از KNN به شما اجازه می‌دهد که شباهت را تنها برای نزدیک‌ترین همسایه‌ها (به عنوان مثال، ۱۰ یا ۲۰ کاربر مشابه) محاسبه کنید، به جای آنکه شباهت بین هر جفت کاربر را در سیستم بررسی کنید. این کار باعث کاهش تعداد محاسبات می‌شود و فقط کاربرانی که از لحاظ شباهت اهمیت بیشتری دارند، در سیستم پیشنهاددهی تأثیر می‌گذارند.

مفهوم (KNN) K-Nearest Neighbors و کاربرد آن در سیستم پیشنهاددهی

KNN یک الگوریتم یادگیری ماشین است که برای دسته‌بندی و رگرسیون استفاده می‌شود و در سیستم‌های پیشنهاددهی برای پیدا کردن کاربران مشابه کاربرد دارد. این الگوریتم با استفاده از شباهت، کاربران نزدیک به کاربر اصلی را شناسایی می‌کند تا از نمرات آن‌ها برای پیش‌بینی و پیشنهاد آیتم‌ها بهره‌برد.

Cosine:

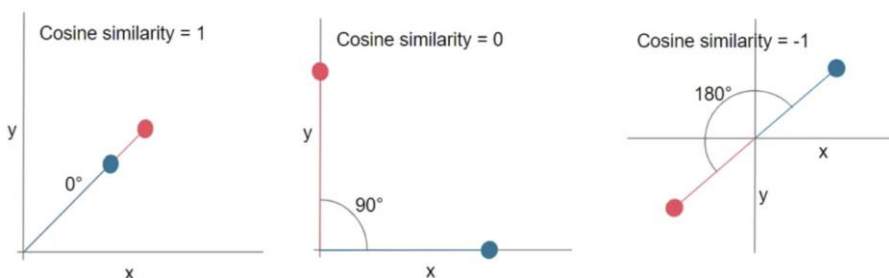
Cosine Similarity یک معیار شباهت است که با اندازه‌گیری زاویه بین دو بردار، میزان شباهت آن‌ها را محاسبه می‌کند. این معیار به‌ویژه در سیستم‌های پیشنهاددهی کاربرد دارد؛ زیرا به ما امکان می‌دهد شباهت بین کاربران یا آیتم‌ها را، بدون توجه به بزرگی مطلق مقادیر، ارزیابی کنیم. مقدار Cosine Similarity بین -۱ و ۱ است؛ هرچه مقدار به ۱ نزدیک‌تر باشد، شباهت بیشتر است.

روش Cosine Similarity: این روش با محاسبه زاویه بین دو بردار نمرات، شباهت بین کاربران را اندازه‌گیری می‌کند.

فرمول آن به صورت:

$$\text{sim}(x, y) = \cos(\theta) = \frac{r_x \cdot r_y}{\|r_x\| \times \|r_y\|}$$

است، که در آن r_x و r_y بردار نمرات دو کاربر هستند.



مشکلات Cosine Similarity ساده:

- این روش ممکن است کاربران را به اشتباه مشابه در نظر بگیرد، چون میانگین نمره‌ها را در نظر نمی‌گیرد.

نحوه محاسبه شباهت کاربران به روش Pearson

این روش شباهت بین کاربران را بر اساس همبستگی (correlation) میان نمرات آن‌ها محاسبه می‌کند و به جای زاویه بین بردارها، رابطه خطی بین نمرات را بررسی می‌کند

مزیت Pearson Correlation

- **در نظر گرفتن میانگین نمرات** : این روش میانگین نمرات کاربران را در محاسبات وارد می‌کند و از این رو برای مقایسه بهتر کاربران با سلايق متفاوت مفید است.
- **تشخیص بهتر شباهت‌ها** : اگر کاربری به صورت کلی نمرات بالاتری نسبت به دیگران دهد، ولی الگوی نمرات مشابهی با کاربر دیگر داشته باشد، Pearson می‌تواند آن‌ها را به عنوان مشابه تشخیص دهد.

مشکلات و محدودیت‌ها

- **حساسیت به داده‌های کمی** : اگر تعداد آیتم‌های مشترک میان دو کاربر کم باشد، محاسبه Pearson ممکن است به نتایج ناپایدار منجر شود.
- **محدودیت در داده‌های پراکنده** : اگر تعداد زیادی از نمرات کاربران خالی یا صفر باشند، نتایج Pearson می‌تواند غیرقابل اعتماد باشد، زیرا بر نمرات مشترک تکیه دارد.

مقایسه Cosine و Pearson

- **Cosine Similarity** فقط زاویه بین بردارها را در نظر می‌گیرد و میانگین نمرات را لحاظ نمی‌کند، بنابراین ممکن است کاربران با میانگین‌های متفاوت ولی الگوی مشابه را به اشتباه متفاوت در نظر بگیرد.
- **Pearson Correlation** با حذف اثر میانگین نمرات، کاربران را بر اساس نحوه‌ی امتیازدهی نسبی‌شان به آیتم‌ها مقایسه می‌کند و برای سیستم‌های پیشنهاددهی که حساسیت بیشتری به سلايق کاربران دارند، مناسب‌تر است.

➤ ساختار کد:

1. بارگزاری داده ها

با اجرای معمولی تابع اجرای csv در pandas که read_csv است، به ارور می خوریم، در اینجا 2 مشکل داریم:

- اینکه DataFrame از "," که جدا کننده پیش فرض است استفاده نکرده و از ";" برای جداسازی استفاده کرده است.
- بعضی از داده ها بصورت اشتباه وارد شده اند و مطابق بقیه در یک ستون جای نگرفته اند.

438640	104943;"3518381768";"6"				
438641	104944;"8871641264";"8"				
438642	104948;"0446310786";"10"				
438643	104951;"0812524861";"7"				
438644	104958;"3257700067";"9"				
438645	104959;"0446343455";"10"				
438646	104959;"0446608262";"8"				
438647	104959;"0449221180";"9"				
438648	104960;"1	488/1	421)"0"		
438649	104962;"8434850915";"8"				
438650	104968;"9024271800";"9"				
438651	104969;"0340792825";"7"				
438652	104976;"0345435168";"6"				
438653	104981;"0352332220";"0"				
438654	104981;"0373764901";"0"				
438655	104981;"0373765460";"0"				
438656	104981;"0590629794";"0"				
438657	104981;"155583227X";"0"				
438658	104981;"1562199064";"0"				
438659	104981;"1573222747";"0"				

برای راه حل ما بجای حذف این داده ها، صرفا در نظر گرفتیم که اشتباه وارد کردن داده ها رخ داده، به همین دلیل separator را در تنظیمات تابع read_csv تصحیح کردیم و در نهایت از کد زیر استفاده کردیم.

```
df = pd.read_csv('Book_Ratings.csv', error_bad_lines=False, warn_bad_lines=True, sep=';')
```

- با استفاده از ' ; ' sep به pandas می گوئید که از ; به عنوان جداکننده استفاده کند.
- پارامتر error_bad_lines=False باعث می شود که خطوطی که تعداد ستون هایشان درست نیست نادیده گرفته شوند و برنامه بدون خطا ادامه یابد.
- پارامتر warn_bad_lines=True پیامی را نمایش می دهد که نشان می دهد کدام خطوط نادیده گرفته شده اند.

با این کار، pandas می تواند ستون ها را به درستی شناسایی کند و ما سه ستون مجزا، User-ID و ISBN و Book-Rating خواهیم داشت.

2. تحلیل اولیه داده ها:

در این بخش اطلاعاتی که می توانیم را از دیتابیس استخراج می کنیم، این اطلاعات در ادامه ذکر شده اند.

- یافتن تعداد کاربران و کتاب های یکتا:

Number of unique users: 105283

Number of unique books: 340556

- محاسبه میانگین و تعداد نمرات برای هر کتاب:

یک DataFrame جدید به نام book_ratings_summary تولید می کنیم که شامل سه ستون است:

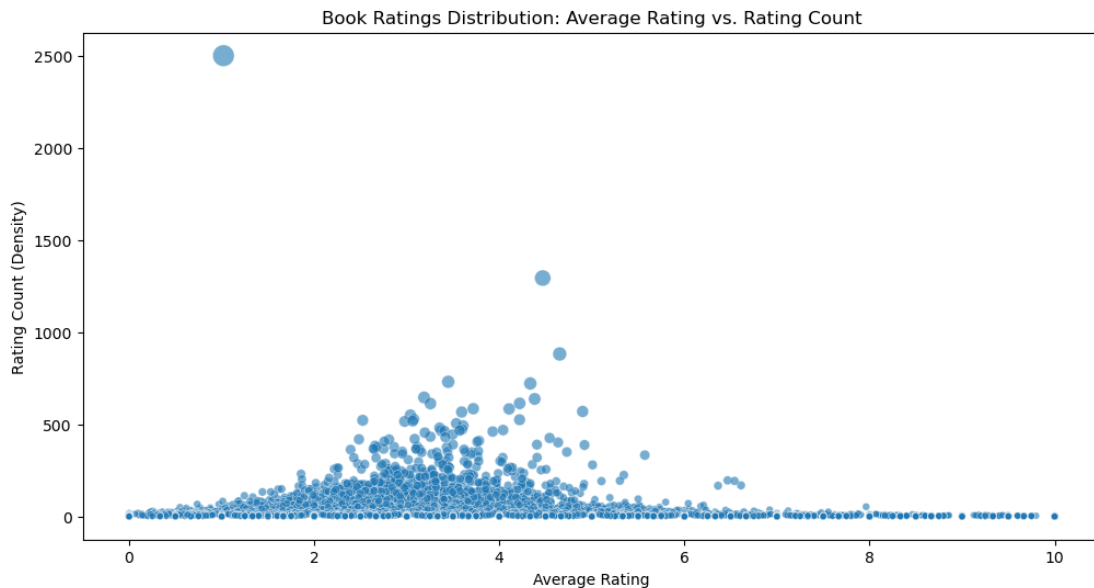
- ISBN: شناسه ی کتاب ها.
- Count: تعداد دفعاتی که هر کتاب نمره دهی شده است.
- Mean: میانگین نمره های داده شده به هر کتاب.

و برای هر کتاب، تعداد دفعات نمره دهی و میانگین نمرات را محاسبه کرده و به صورت یک DataFrame جدید ذخیره می کند.

	A
1	User-ID;"ISBN";"Book-Rating"
2	276725;"034545104X";"0"
3	276726;"0155061224";"5"
4	276727;"0446520802";"0"
5	276729;"052165615X";"3"
6	276729;"0521795028";"6"
7	276733;"2080674722";"0"
8	276736;"3257224281";"8"
9	276737;"0600570967";"6"
10	276744;"038550120X";"7"

• رسم نمودار نمره‌های کتاب‌ها:

بررسی تعداد نمره‌های داده شده به کتاب‌ها، و مقدار این نمرات



- `y='Rating_Count'` و `x='Average_Rating'`: محور افقی میانگین نمره هر کتاب و محور عمودی تعداد نمره‌ها یا چگالی هر کتاب را نشان می‌دهد.
- `size='Rating_Count'`: اندازه نقاط متناسب با تعداد دفعات نمره‌دهی به هر کتاب است؛ کتاب‌هایی که تعداد بیشتری نمره دریافت کرده‌اند، نقاط بزرگ‌تری خواهند داشت.
- `sizes=(20, 200)`: اندازه‌ی نقاط را در یک محدوده مشخص تنظیم می‌کند تا تفاوت اندازه‌ها قابل مشاهده باشد.
- `alpha=0.6`: شفافیت نقاط را کمی کاهش می‌دهد تا نقاط متراکم بهتر دیده شوند.
- `legend=False`: حذف نمایش اندازه‌ها در راهنمای کنار نمودار (می‌توان آن را تنظیم کرد تا نمایش داده شود).

این نمودار به شما کمک می‌کند تا کتاب‌هایی که تعداد نمرات بالایی دارند (چگالی بیشتر) و همچنین میانگین نمره بالاتری دارند را به راحتی شناسایی کنید.

1. توزیع کلی میانگین نمرات:

- اکثر کتاب‌ها میانگین نمرات پایین دارند و بیشتر نمرات در محدوده‌ی ۰ تا ۴ قرار دارند.
- کتاب‌هایی با میانگین نمره‌ی بالا (بالاتر از ۸) کم‌تعداد هستند و به ندرت دیده می‌شوند. این موضوع نشان‌دهنده‌ی این است که یا کاربران عموماً به کتاب‌ها نمرات پایینی می‌دهند یا اینکه کتاب‌های با نمره‌ی بالا محدود هستند.

2. رابطه بین تعداد نمرات (چگالی) و میانگین نمره:

- کتاب‌هایی که بیشترین تعداد نمره‌ها (چگالی بالا) را دریافت کرده‌اند، میانگین نمره پایینی دارند. برای مثال، کتابی که تعداد نمره‌دهی بالاتر از ۲۵۰۰ دارد، میانگین نمره پایینی در حدود ۲ دارد.
- این می‌تواند نشان دهد که کتاب‌های پرفروش یا کتاب‌هایی که بیشتر شناخته شده‌اند، به صورت متوسط نمره‌های کمتری از کاربران دریافت کرده‌اند. این اتفاق ممکن است به این دلیل باشد که کتاب‌های معروف‌تر بیشتر در دسترس کاربران با سلیقه‌های مختلف قرار می‌گیرند، که برخی ممکن است نمره پایین‌تری بدهند.

3. پراکندگی داده‌ها:

- بیشتر داده‌ها در بخش پایین نمودار قرار دارند (یعنی تعداد نمرات کمتر از ۵۰۰)، اما پراکندگی قابل توجهی وجود دارد.
- این موضوع نشان‌دهنده‌ی این است که تعداد زیادی از کتاب‌ها وجود دارند که نمره‌دهی‌های نسبتاً کمی دارند.
- همچنین، برخی از کتاب‌ها با تعداد نمرات بالا و میانگین‌های مختلف وجود دارند که به عنوان نقاط برجسته در نمودار دیده می‌شوند.

4. کتاب‌های با میانگین نمره بالا اما چگالی کم:

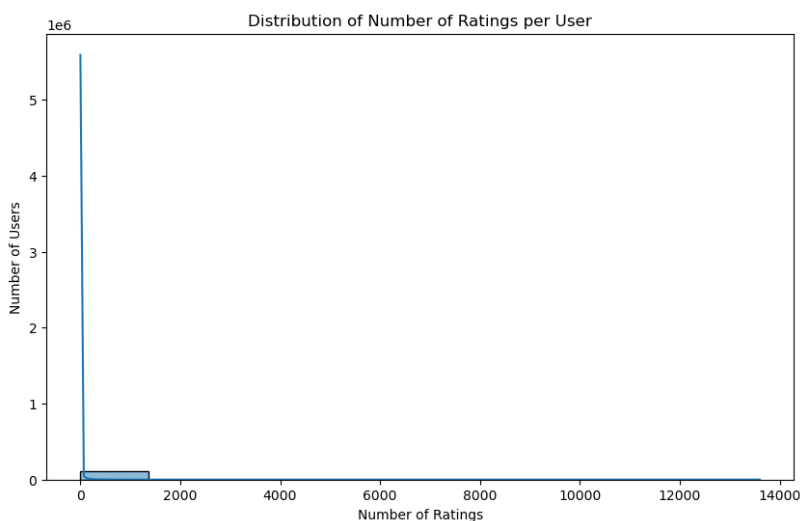
- در سمت راست نمودار، نقاطی با میانگین نمره بالا ولی تعداد نمره‌دهی پایین وجود دارند. این کتاب‌ها احتمالاً کتاب‌های خاص و کمتر شناخته‌شده‌ای هستند که تنها توسط تعداد کمی از کاربران نمره‌دهی شده‌اند. ممکن است این کتاب‌ها مخاطب خاص‌تری داشته باشند و به همین دلیل نمرات بالاتری از این کاربران خاص دریافت کرده باشند.

به طور خلاصه، این نمودار به ما نشان می‌دهد که کتاب‌های با چگالی بالا معمولاً میانگین نمره‌ی پایینی دارند و بیشتر کتاب‌ها نمرات متوسط به پایین گرفته‌اند.

یعنی این کتابفروشی خیلیم کتابای خوبی نداره:)

• رسم نمودار توزیع تعداد نمرات داده شده توسط کاربران

برای این کار، می‌توانیم از نمودار هیستوگرام یا نمودار توزیع استفاده کنیم که تعداد امتیازدهی‌ها را برای هر کاربر نشان می‌دهد.



نتایج این نمودار نشان می‌دهد که توزیع تعداد نمرات کاربران به شدت چگالی محور و نامتقارن است، به این معنا که بیشتر کاربران تعداد بسیار کمی امتیاز داده‌اند. در اینجا چند نکته قابل توجه است:

1. تمرکز بیشتر کاربران در تعداد نمرات کم:

- تقریباً همه‌ی کاربران در بازه‌ی سمت چپ نمودار قرار دارند که نشان می‌دهد بیشتر کاربران تنها به چند کتاب امتیاز داده‌اند (برای مثال، بین ۱ تا ۱۰ نمره).
- تعداد کمی از کاربران به تعداد زیادی کتاب نمره داده‌اند. این نشان می‌دهد که اکثر کاربران به شکل غیر فعال‌تری از سیستم استفاده می‌کنند و تنها تعداد کمی از کاربران فعالانه به تعداد زیادی کتاب امتیاز داده‌اند.

2. تعداد بسیار کم کاربران با نمرات بالا (کاربران فعال):

- نمودار دارای یک دم طولانی است، که به این معنی است که تعداد بسیار کمی از کاربران به تعداد بسیار بالایی از کتاب‌ها نمره داده‌اند (چندین هزار نمره). این کاربران احتمالاً از کاربران فعال و علاقه‌مند هستند که به عنوان **کاربران کلیدی** برای تحلیل‌های بیشتر و الگوریتم‌های پیشنهاددهی می‌توانند مهم باشند.

3. پراکندگی بالا و عدم یکنواختی:

- این توزیع نشان‌دهنده‌ی **پراکندگی بالا** در رفتار کاربران است؛ بسیاری از کاربران فقط به تعداد محدودی کتاب نمره داده‌اند، در حالی که تعداد معدودی از کاربران به طور مداوم به کتاب‌های بیشتری نمره داده‌اند.

4. نیاز به فیلتر کردن کاربران کم‌فعال:

- با توجه به اینکه اکثر کاربران تعداد کمی نمره داده‌اند، می‌توان کاربرانی که زیر یک آستانه مشخص (مثلاً ۳ یا ۵ نمره) امتیاز داده‌اند را فیلتر کرد. این کار می‌تواند به کاهش حجم داده‌ها و افزایش کارایی در محاسبات کمک کند، مخصوصاً در هنگام استفاده از روش‌های Collaborative Filtering.

3. پردازش داده ها:

به دلیل حجم بالای داده های ورودی مجبور به انجام مراحلی بودیم، که به مشکل حافظه (cache) بر نخوریم و سرعت هم کند نشود.

✓ **فیلترینگ:** فیلتر کردن کاربران و کتاب های غیر فعال به کاهش حجم داده ها و پیچیدگی محاسبات کمک می کند. کاربران و کتاب هایی با نمرات کم معمولاً اطلاعات کافی برای سیستم پیشنهاددهی ندارند؛ با حذف آن ها، سیستم سریع تر و دقیق تر عمل می کند.

```
active_users = user_counts[user_counts >= 10].index
df_active = df[df['User-ID'].isin(active_users)].copy()
```

این دو خط کد کاربران فعال را فیلتر می کند، یعنی کاربرانی که حداقل ۱۰ فعالیت (امتیازدهی) داشته اند. ابتدا شناسه کاربران فعال با ۱۰ فعالیت یا بیشتر استخراج می شود و سپس فقط ردیف های مربوط به این کاربران در دیتافریم جدید (df_active) نگه داشته می شود.

✓ **ماتریس پراکنده:** ماتریس پراکنده (Sparse Matrix) ماتریسی است که بیشتر عناصر آن صفر یا خالی هستند و تنها تعداد کمی از خانه های آن دارای مقادیر معتبر می باشند که ما هر کاربر را به عنوان یک بردار در نظر می گیریم. این نوع ماتریس در سیستم های پیشنهاددهی کاربرد زیادی دارد؛ زیرا کاربران معمولاً به تعداد محدودی از آیتم ها (مانند کتاب ها یا فیلم ها) امتیاز می دهند. بنابراین، ماتریس امتیازدهی کاربران به آیتم ها معمولاً پراکنده است

در ساختار فعلی، هر ردیف نشان دهنده نمره یک کاربر به یک کتاب خاص است؛ بنابراین، هر کاربر می تواند چندین بار با کتاب های مختلف در ردیف ها ظاهر شود. این ساختار برای تحلیل داده ها انعطاف پذیر است، اما برای محاسبات شباهت و یافتن همسایه ها بهینه نیست.

در ماتریس پراکنده ای که برای استفاده در KNN ساخته می شود، هر کاربر با بردار نمرات خود به کتاب ها در یک ردیف مجزا قرار می گیرد. این ماتریس به گونه ای طراحی شده که تنها مقادیر غیر صفر ذخیره شوند، بنابراین محاسبات بهینه تر انجام می گیرد.

ویژگی ها و مزایا:

- **حافظه کمتر:** ماتریس های پراکنده در حافظه به طور بسیار بهینه تری نسبت به ماتریس های معمولی ذخیره می شوند، زیرا فقط خانه های غیر صفر را ذخیره می کنند.
- **دسترسی سریع:** به دلیل ساختار ساده تری که دارند، دسترسی به داده های مورد نیاز در آن ها سریع تر انجام می شود، که در الگوریتم های سیستم های پیشنهاددهی بهینه است.
- **پشتیبانی از الگوریتم ها:** بسیاری از الگوریتم ها، مانند KNN و مدل های یادگیری ماشین، می توانند به طور مستقیم با ماتریس های پراکنده کار کنند، که محاسبات را سریع تر و بهینه تر می کند.

از اینجا به بعد مراحل پیاده سازی cosine و pearson متفاوت می شود، بنابراین ابتدا به روش cosine می پردازیم، و سپس pearson را توضیح می دهیم.

Cosine:

4. پیدا کردن همسایه ها به روش KNN با Cosine:

○ ساخت مدل KNN:

- در این قسمت، مدل KNN با استفاده از Cosine Similarity ساخته می شود.
- NearestNeighbors با `metric='cosine'` و `n_neighbors=5` تنظیم شده که برای هر کاربر، 5 نزدیکترین همسایه را بر اساس فاصله کساین پیدا می کند.
- `sparse_matrix`، که ماتریس کاربران و نمرات آن هاست، برای آموزش مدل استفاده می شود.

○ تابع `find_nearest_neighbors`:

- **کارکرد:** پیدا کردن نزدیکترین همسایه ها برای یک کاربر مشخص.
- این تابع از `kneighbors` مدل KNN استفاده می کند تا نزدیکترین همسایه ها و فاصله آن ها با بردار کاربر را پیدا کند.
- برای هر همسایه، `(index, distance)` را به لیست `neighbors` اضافه می کند.
- نتایج شامل ایندکس کاربران و فاصله کساین آن هاست که می تواند برای توصیه ها استفاده شود.

در این مثال از الگوریتم brute استفاده کردیم چون Cosine Similarity را به عنوان متریک شباهت انتخاب کرده بودیم. الگوریتم brute برای Cosine Similarity مناسب است، چرا که مستقیماً و به صورت کامل فاصله ها را محاسبه می کند و با داده های پراکنده (Sparse) سازگاری خوبی دارد.

الگوریتم های دیگر موجود در KNN:

scikit-learn چهار الگوریتم مختلف برای KNN ارائه می دهد که هر کدام کاربرد خاص خود را دارند:

- **Brute:**

برای ابعاد بالا و متریک هایی مانند Cosine Similarity مناسب است.

- **Ball Tree , KD Tree:**

برای داده های پیوسته و با ابعاد کمتر مناسب تر هستند و معمولاً برای متریک های فاصله مانند Euclidean و Manhattan به کار می روند.

- **Auto:**

اگر ندانید کدام الگوریتم برای داده های شما مناسب است، از این گزینه استفاده کنید، اما اگر متریک خاصی مانند Cosine دارید، بهتر است الگوریتم را به طور دستی مشخص کنید.

تابع `find_nearest_neighbors_advanced`:

این تابع `find_nearest_neighbors_advanced` به منظور یافتن نزدیک‌ترین همسایگان برای یک وکتور کاربر خاص با استفاده از یک مدل KNN طراحی شده است. عملکرد این تابع به صورت زیر است:

عملکرد تابع:

1. ورودی‌ها:
 - `knn_model`: مدل KNN از پیش آموزش‌دیده که برای محاسبه نزدیکی استفاده می‌شود.
 - `user_vector`: وکتور کاربر که نماینده ویژگی‌های کاربر است.
 - `n_neighbors`: تعداد نزدیک‌ترین همسایگان مورد نظر برای بازیابی.
 - `existing_user`: یک پرچم که نشان می‌دهد آیا کاربر در سیستم وجود دارد یا خیر.
 2. درخواست همسایگان اضافی:
 - برای اطمینان از اینکه حداقل تعداد همسایگان مورد نظر بازیابی می‌شود، تعداد همسایگان اضافی درخواست می‌شود.
 3. محاسبه فاصله و شباهت:
 - فاصله‌ها و ایندکس‌های همسایگان محاسبه می‌شود و سپس فاصله به شباهت تبدیل می‌شود.
 4. حذف همسایگان مشابه:
 - اگر کاربر در سیستم وجود داشته باشد و شباهت یک همسایه بیشتر از 0.9999 باشد، آن همسایه نادیده گرفته می‌شود.
 5. جمع‌آوری نتایج:
 - اطلاعات همسایگان شامل ایندکس، شباهت و فاصله جمع‌آوری و به لیست اضافه می‌شود.
 6. نمایش همسایگان:
 - همسایگان انتخاب‌شده با نمایش ایندکس کاربر، فاصله و شباهت چاپ می‌شوند.
 7. خروجی:
 - لیستی از همسایگان نزدیک به کاربر که شامل ایندکس، شباهت و فاصله است، به تابع برگردانده می‌شود.
- این تابع به طور خاص به کاربران با شباهت بالا و همسایگان نزدیک‌تر توجه می‌کند و از مصرف بی‌مورد داده‌ها جلوگیری می‌کند.

5. پیشنهادات کتاب:

○ تابع `recommend_books_optimized`:

- کارکرد: توصیه کتاب‌ها به کاربر بر اساس نمرات نزدیک‌ترین همسایه‌ها.
- ابتدا کتاب‌هایی که کاربر هنوز نمره نداده شناسایی می‌شوند. (`unrated_books`)
- سپس، برای هر کتاب، نمرات همسایه‌ها به آن کتاب دریافت و میانگین وزنی محاسبه می‌شود:

```
weighted_sum = np.dot(similarities[valid_ratings_mask], neighbor_ratings[valid_ratings_mask])
similarity_sum = np.sum(similarities[valid_ratings_mask])
```

- نمرات پیش‌بینی‌شده در `book_scores` ذخیره می‌شود و در نهایت `top_n` کتاب با بالاترین نمره برگردانده می‌شوند.

○ تابع `display_recommended_books`:

```
def display_recommended_books(recommended_books, book_mapping):
    index_to_isbn = {index: isbn for isbn, index in book_mapping.items()}

    print("Recommended Books:")
    for book_index, predicted_rating in recommended_books:
        isbn = index_to_isbn.get(book_index, "Unknown ISBN")

        print(f"ISBN: {isbn}, Predicted Rating: {predicted_rating:.4f}")
```

- کارکرد: نمایش کتاب‌های توصیه‌شده با ISBN، عنوان و نمره پیش‌بینی‌شده.
- ابتدا `book_mapping` معکوس می‌شود تا `index` کتاب به ISBN تبدیل شود.
- سپس ISBN هر کتاب نمایش داده می‌شود.

چالش:

```
<class 'scipy.sparse._csr.csr_matrix'>
Neighbor 1: User Index = 0, Cosine Distance = 0.0
Neighbor 2: User Index = 8190, Cosine Distance = 0.8280814054482752
Neighbor 3: User Index = 9689, Cosine Distance = 0.8590603976058879
Neighbor 4: User Index = 618, Cosine Distance = 0.8794084415072492
Neighbor 5: User Index = 6863, Cosine Distance = 0.9186729768566776
Recommended Books:
ISBN: 0679731148, Predicted Rating: 10.0000
ISBN: 1898307148, Predicted Rating: 10.0000
ISBN: 0804119716, Predicted Rating: 10.0000
ISBN: 0804119724, Predicted Rating: 10.0000
ISBN: 1575666065, Predicted Rating: 10.0000
```

هرچه فاصله به 1 نزدیک تر باشد میزان شباهت کمتر است. در اینجا با توجه به اینکه شباهت کمی داریم، اما در پیش‌بینی، گفتیم که نمره کامل خواهد داد.

این مشکل به دلیل نحوه محاسبات در **Cosine** است. برای همین از **Mean-Centered Cosine Similarity** استفاده می‌کنیم.

Mean-Centered Cosine Similarity

Cosine Similarity، به تنهایی، همیشه به طور دقیق برای داده‌های امتیازدهی (مانند نمرات کتاب‌ها) کارآمد نیست، زیرا Cosine Similarity تنها جهت دو بردار را در نظر می‌گیرد و نه مقادیر آن‌ها را نسبت به میانگین. این می‌تواند منجر به نتایج نامعقولی شود، به خصوص اگر مقادیر نمرات کاربران به شکل متفاوتی توزیع شده باشد. در نتیجه، حتی اگر Cosine Distance بالا باشد (یعنی شباهت پایین)، ممکن است نمرات پیش‌بینی‌شده غیرواقعی و بالا باشند. این کار نرمال‌سازی داده‌ها را انجام می‌دهد و اثر مقادیر مطلق امتیازات را کاهش می‌دهد.

پیاده‌سازی Mean-Centered Cosine Similarity

برای پیاده‌سازی این روش، مراحل زیر را انجام می‌دهیم:

1. محاسبه میانگین نمرات هر کاربر.
2. کسر میانگین هر کاربر از نمرات آن کاربر برای ایجاد ماتریس Mean-Centered.
3. محاسبه شباهت کسینوسی با استفاده از این ماتریس Mean-Centered.

```
Neighbor 1: User Index = 8190, Similarity = 0.171919850056095
Neighbor 2: User Index = 9689, Similarity = 0.14093859028951372
Neighbor 3: User Index = 618, Similarity = 0.12059046488889613
Neighbor 4: User Index = 6863, Similarity = 0.08132626586646463
Neighbor 5: User Index = 8231, Similarity = 0.07842784398412683
Neighbor 1 (User Index: 8190) has rated books and scores: [(2, 9.0), (3814, 9.0), (20496, 7.0), (22020, 7.0), (47189, 5.0), (72010, 8.0), (
Neighbor 2 (User Index: 9689) has rated books and scores: [(2, 5.0), (9754, 7.0), (11720, 6.0), (11958, 7.0), (26720, 6.0), (28698, 9.0)]
Neighbor 3 (User Index: 618) has rated books and scores: [(2, 5.0), (673, 5.0), (1209, 8.0), (1289, 8.0), (2767, 5.0), (10426, 8.0), (16927
Neighbor 4 (User Index: 6863) has rated books and scores: [(2, 8.0), (1278, 8.0), (1327, 10.0), (1736, 9.0), (4545, 6.0), (7587, 8.0), (862
Neighbor 5 (User Index: 8231) has rated books and scores: [(2, 7.0), (90, 8.0), (687, 8.0), (1209, 7.0), (1222, 7.0), (1651, 8.0), (1669, 7
Results saved to neighbor_nonzero_ratings_with_scores.txt
book_scores: {90: 8.0, 673: 5.0, 687: 8.0, 1209: 7.605926487727493, 1222: 7.0, 1278: 8.0, 1289: 8.0, 1327: 10.0, 1651: 8.0, 1669: 7.0, 173
Recommended Books:
ISBN: 0679731148, Predicted Rating: 10.0000
ISBN: 1898307148, Predicted Rating: 10.0000
ISBN: 0804119716, Predicted Rating: 10.0000
ISBN: 0804119724, Predicted Rating: 10.0000
ISBN: 1575666065, Predicted Rating: 10.0000
```

این ایده می‌تواند به بهبود نتایج کمک کند. اگر کتابی بین همسایگان پرتکرار باشد و نمره‌های بیشتری از همسایگان گرفته باشد، می‌تواند نشان‌دهنده محبوبیت یا کیفیت بالاتر آن کتاب باشد. با استفاده از این روش، ما می‌توانیم اولویت بیشتری به کتاب‌هایی بدهیم که هم پرتکرار هستند و هم نمرات بالایی دارند.

نتایج با تابع frequency: (این تابع در ادامه معرفی شده است.)

```
Neighbor 1: User Index = 8190, Similarity = 0.171919850056095
Neighbor 2: User Index = 9689, Similarity = 0.14093859028951372
Neighbor 3: User Index = 618, Similarity = 0.12059046488889613
Neighbor 4: User Index = 6863, Similarity = 0.08132626586646463
Neighbor 5: User Index = 8231, Similarity = 0.07842784398412683
Results saved to neighbor_nonzero_ratings_with_scores.txt
Recommended Books:
ISBN: 0440241073, Predicted Rating: 7.6059, Frequency: 2
ISBN: 055356160X, Predicted Rating: 6.9819, Frequency: 2
ISBN: 0679731148, Predicted Rating: 10.0000, Frequency: 1
ISBN: 1898307148, Predicted Rating: 10.0000, Frequency: 1
ISBN: 0804119716, Predicted Rating: 10.0000, Frequency: 1
```

در ادامه روش pearson را توضیح می‌دهیم.

Pearson:

6. پیدا کردن همسایه‌ها

برای محاسبه همبستگی پیرسون می‌توان از کتابخانه‌ها و روش‌های مختلف استفاده کرد. ابتدا از کد زیر استفاده کردیم:

```
correlation_distances = 1 - squareform(pdist(dense_matrix, metric='correlation'))
knn_model = NearestNeighbors(metric='precomputed', algorithm='brute', n_neighbors=5, n_jobs=-1)
knn_model.fit(correlation_distances)
```

چالش:

اما با ارور زیر مواجه شدیم:

MemoryError: Unable to allocate 29.9 GiB for an array with shape (13097, 306779) and data type float64

با اینکه از قابلیت sparse برای کاهش مصرف حافظه استفاده کردیم، محاسبه کل ماتریس فاصله برای تمامی جفت‌های کاربران به صورت dense (full matrix) انجام می‌شود که به مقدار زیادی حافظه نیاز دارد. این موضوع به‌خصوص زمانی که تعداد کاربران و کتاب‌ها زیاد باشد، منجر به ارور MemoryError می‌شود. در واقع، در حالی که sparse به ذخیره‌سازی کارآمدتر داده‌ها کمک می‌کند، محاسبه فاصله‌ها به صورت full matrix با نوع داده float64 باعث افزایش شدید مصرف حافظه می‌شود.

راه‌حل بعدی:

به عنوان راه‌حل، از کد زیر استفاده کردیم:

```
knn_model = NearestNeighbors(metric='correlation', algorithm='brute', n_neighbors=5, n_jobs=-1)
knn_model.fit(sparse_matrix.toarray())
```

در این کد از محاسبه‌ی فاصله‌ها به صورت on-the-fly استفاده می‌شود. با تنظیم metric='correlation' در مدل NearestNeighbors، فاصله‌ی همبستگی پیرسون تنها زمانی محاسبه می‌شود که به آن نیاز است و نیازی به ذخیره‌سازی کل ماتریس فاصله نیست. این روش به دلیل عدم نیاز به نگهداری کل ماتریس در حافظه، حافظه‌ی کمتری مصرف می‌کند و تمرکز آن تنها بر روی داده‌های ورودی است. به این ترتیب، محاسبه‌ی فاصله‌ها فقط در زمان نیاز و برای جفت‌های مشخص انجام می‌شود.

چالش:

با این حال، باز هم مشکل حافظه خوردیم:

MemoryError: Unable to allocate 29.9 GiB for an array with shape (13097, 306779) and data type float64

راه حل بعدی:

استفاده از مدل های Approximate Nearest Neighbor:

مدل های Approximate Nearest Neighbor مانند Faiss و Annoy برای کاهش مصرف حافظه و افزایش سرعت طراحی شده اند. این کتابخانه ها می توانند در پردازش داده های بزرگ بسیار مؤثر باشند، اما نکات کلیدی زیر باید در نظر گرفته شود:

- پشتیبانی از متریک پیرسون Faiss و Annoy به طور مستقیم از متریک همبستگی پیرسون پشتیبانی نمی کنند. این کتابخانه ها عمدتاً برای فاصله های کسینوسی، و دیگر متریک های پرکاربرد طراحی شده اند.
 - Annoy از فاصله کسینوسی و مانهتن (L1) پشتیبانی می کند، اما متریک پیرسون به طور مستقیم موجود نیست.
 - Faiss نیز به طور پیش فرض از متریک های معمول مانند L2 و کسینوسی پشتیبانی می کند.

چالش:

اگر دقت بالای همبستگی پیرسون برای کاربرد شما حیاتی باشد، ممکن است نیاز به استفاده از روش هایی برای تقریبی سازی این متریک در Faiss یا Annoy باشد. یکی از روش های ساده، مرکزی سازی داده ها (کسر میانگین هر کاربر از امتیازات) و سپس استفاده از کسینوس مشابهت است که معمولاً به همبستگی پیرسون نزدیک می شود.

مشکل در استفاده: استفاده از این کتابخانه ها به دلیل عدم پشتیبانی از متریک پیرسون و همچنین مشکلات مربوط به تنظیم ورژن های ناپای و پانداس در محیط Visual Studio Code با C++، به چالش کشیده شد و موجب شد نتوان از آن ها بهره برداری کرد.

راه حل بعدی:

کاهش ابعاد داده ها

برای کاهش مصرف حافظه در داده های بزرگ، می توان از تکنیک های کاهش ابعاد مانند Truncated SVD استفاده کرد، که به ویژه برای داده های پراکنده طراحی شده است.

این روش به طور کلی به بهبود کارایی و کاهش نیاز به منابع حافظه کمک می کند. پس از استفاده از Truncated SVD، داده ها به صورت فشرده شده و با ابعاد کمتر ذخیره می شوند، در حالی که بخش زیادی از اطلاعات اصلی داده ها حفظ شده است. این امر باعث می شود که مشکل حافظه برای محاسبه KNN و همچنین سرعت محاسبات بهبود یابد.

7. توابع پیشنهاد کتاب

2 تابع زیر برای پیشنهاد دهی پیاده سازی شده اند:

1. `recommend_books_optimized`:

- کتاب‌هایی را که کاربر نمره نداده، شناسایی می‌کند.
- نمره پیش‌بینی شده هر کتاب را بر اساس نمرات و شباهت همسایگان محاسبه می‌کند.
- کتاب‌ها را بر اساس نمره پیش‌بینی شده مرتب کرده و ۵ کتاب برتر را برمی‌گرداند.

2. `recommend_books_with_frequency`:

- این تابع برای توصیه کتاب‌ها به یک کاربر خاص طراحی شده است و هم به نمرات پیش‌بینی شده و هم به تعداد دفعاتی که هر کتاب توسط همسایگان امتیازدهی شده است توجه می‌کند.
- کتاب‌ها بر اساس تعداد دفعات نمره‌دهی توسط همسایگان و سپس بر اساس نمرات پیش‌بینی شده مرتب می‌شوند.
- پنج کتاب با بالاترین ترکیب نمره و فراوانی به عنوان پیشنهاد بازمی‌گردند.

تابع ذخیره‌سازی نمرات غیرصفر:

`save_nonzero_ratings_per_neighbor`:

- فهرستی از کتاب‌هایی را که هر همسایه به آن‌ها نمره غیرصفر داده است، همراه با نمره‌ها و شباهت/فاصله‌ی هر همسایه، در یک فایل متنی ذخیره می‌کند.
- این تابع اجازه می‌دهد تا بررسی کنید که هر همسایه به کدام کتاب‌ها نمره داده و با چه نمراتی.

نتیجه‌گیری

- بهترین روش: در بیشتر موارد، `recommend_books_with_mask` می‌تواند دقیق‌تر باشد، چون از ماسک برای متمایز کردن نمرات واقعی استفاده می‌کند که احتمالاً پردازش را دقیق‌تر می‌کند.
- روش‌های جایگزین: اگر سادگی و سرعت مهم‌تر از دقت است، `recommend_books_ignore_zeros` گزینه مناسبی است، زیرا مستقیماً نمرات صفر را حذف می‌کند و بدون نیاز به ماسک کار می‌کند.
- مقیاس‌بندی نمایی: در هر سه تابع، مقیاس‌بندی نمایی شباهت‌ها به همسایگان با شباهت بالا وزن بیشتری می‌دهد که به بهبود دقت پیش‌بینی کمک می‌کند.

خلاصه کارهای انجام شده:

مراحل کار در سیستم پیشنهاد دهنده:

User-Based Collaborative Filtering:

1. ساخت ماتریس امتیازدهی کاربران-آیتم‌ها.
2. پیدا کردن نزدیک‌ترین همسایه‌ها برای هر کاربر.
3. محاسبه شباهت بین کاربران:
 - Cosine Similarity
 - Pearson Correlation
4. پیش‌بینی نمرات و پیشنهاد آیتم‌ها.

Cosine Implementation:

1. ساخت مدل KNN با 'metric='cosine'.
2. پیاده‌سازی تابع find_nearest_neighbors.
3. محاسبه نزدیک‌ترین همسایگان.
4. پیاده‌سازی تابع recommend_books_optimized.

Pearson Implementation:

1. محاسبه ماتریس فاصله همبستگی با pdist.
2. ساخت مدل KNN با 'metric='correlation'.
3. پیاده‌سازی تابع find_nearest_neighbors برای پیدا کردن همسایه‌ها.
4. پیش‌بینی نمرات با تابع recommend_books_optimized.

Mean-Centered Cosine Implementation:

1. محاسبه میانگین نمرات هر کاربر.
2. کسر میانگین از نمرات برای ایجاد ماتریس Mean-Centered.
3. محاسبه شباهت کسینوسی با ماتریس Mean-Centered.
4. استفاده از نتایج برای پیش‌بینی نمرات و پیشنهاد آیتم‌ها.

نتایج:

نتایج زیر برای کاربر شماره 0 است.

Cosine:

```
Cosine similarity!
Neighbor 1: User Index = 8190, Similarity = 0.17191859455172476
Neighbor 2: User Index = 9689, Similarity = 0.14093960239411207
Neighbor 3: User Index = 618, Similarity = 0.12059155849275083
Neighbor 4: User Index = 6863, Similarity = 0.08132702314332241
Neighbor 5: User Index = 8231, Similarity = 0.07842764352626463
neighbors: [(8190, 0.17191859455172476), (9689, 0.14093960239411207), (618, 0.12059155849275083), (6863, 0.08132702314332241), (8231, 0.07842764352626463)]
Neighbor 1 (User Index: 8190) has rated books and scores: [(2, 9), (3814, 9), (20496, 7), (22020, 7), (47189, 5), (72010, 8), (78365, 5), (87472, 7), (131581, 8), (210903, 5)]
Neighbor 2 (User Index: 9689) has rated books and scores: [(2, 5), (9754, 7), (11720, 6), (11958, 7), (26720, 6), (28698, 9)]
Neighbor 3 (User Index: 618) has rated books and scores: [(2, 5), (673, 5), (1209, 8), (1289, 8), (2767, 5), (10426, 8), (16927, 6), (39944, 7), (39945, 5)]
Neighbor 4 (User Index: 6863) has rated books and scores: [(2, 8), (1278, 8), (1327, 10), (1736, 9), (4545, 6), (7587, 8), (8623, 9), (8650, 6), (19695, 10), (29561, 5), (31204, 6)]
Neighbor 5 (User Index: 8231) has rated books and scores: [(2, 7), (90, 8), (687, 8), (1209, 7), (1222, 7), (1651, 8), (1669, 7), (1915, 6), (2013, 7), (2323, 8), (2650, 6)]
Results saved to neighbor_nonzero_ratings_with_scores.txt
Recommended Books:
ISBN: 0440241073, Predicted Rating: 7.6059, Frequency: 2
ISBN: 055356160X, Predicted Rating: 6.9819, Frequency: 2
ISBN: 0679731148, Predicted Rating: 10.0000, Frequency: 1
ISBN: 1898307148, Predicted Rating: 10.0000, Frequency: 1
ISBN: 0804119716, Predicted Rating: 10.0000, Frequency: 1
```

Mean centered cosine:

```
Mean Centered Cosine similarity!
Neighbor 1: User Index = 8190, Similarity = 0.171919850056095
Neighbor 2: User Index = 9689, Similarity = 0.14093859028951372
Neighbor 3: User Index = 618, Similarity = 0.1205904648889613
Neighbor 4: User Index = 6863, Similarity = 0.08132626586646463
Neighbor 5: User Index = 8231, Similarity = 0.07842784398412683
neighbors: [(8190, 0.171919850056095), (9689, 0.14093859028951372), (618, 0.1205904648889613), (6863, 0.08132626586646463), (8231, 0.07842784398412683)]
Neighbor 1 (User Index: 8190) has rated books and scores: [(2, 9.0), (3814, 9.0), (20496, 7.0), (22020, 7.0), (47189, 5.0), (72010, 8.0), (78365, 5.0), (87472, 7.0), (131581, 8.0), (210903, 5.0)]
Neighbor 2 (User Index: 9689) has rated books and scores: [(2, 5.0), (9754, 7.0), (11720, 6.0), (11958, 7.0), (26720, 6.0), (28698, 9.0)]
Neighbor 3 (User Index: 618) has rated books and scores: [(2, 5.0), (673, 5.0), (1209, 8.0), (1289, 8.0), (2767, 5.0), (10426, 8.0), (16927, 6.0), (39944, 7.0), (39945, 5.0)]
Neighbor 4 (User Index: 6863) has rated books and scores: [(2, 8.0), (1278, 8.0), (1327, 10.0), (1736, 9.0), (4545, 6.0), (7587, 8.0), (8623, 9.0), (8650, 6.0), (19695, 10.0), (29561, 5.0), (31204, 6.0)]
Neighbor 5 (User Index: 8231) has rated books and scores: [(2, 7.0), (90, 8.0), (687, 8.0), (1209, 7.0), (1222, 7.0), (1651, 8.0), (1669, 7.0), (1915, 6.0), (2013, 7.0), (2323, 8.0), (2650, 6.0)]
Results saved to neighbor_nonzero_ratings_with_scores.txt
Recommended Books:
ISBN: 0440241073, Predicted Rating: 7.6059, Frequency: 2
ISBN: 055356160X, Predicted Rating: 6.9819, Frequency: 2
ISBN: 0679731148, Predicted Rating: 10.0000, Frequency: 1
ISBN: 1898307148, Predicted Rating: 10.0000, Frequency: 1
ISBN: 0804119716, Predicted Rating: 10.0000, Frequency: 1
```

Pearson:

```
Pearson similarity!
Neighbor 1: User Index = 3714, Similarity = 0.7531
Neighbor 2: User Index = 1913, Similarity = 0.7497
Neighbor 3: User Index = 9877, Similarity = 0.7490
Neighbor 4: User Index = 12173, Similarity = 0.7482
Neighbor 5: User Index = 7706, Similarity = 0.7470
neighbors: [(3714, 0.2469063952124556), (1913, 0.2502736679663742), (9877, 0.2510237331738202), (12173, 0.2518481523919395), (7706, 0.2530058319176587)]
Neighbor 1 (User Index: 3714) has rated books and scores: [(4701, 2), (43717, 5), (72188, 3), (131163, 6), (131164, 5)]
Neighbor 2 (User Index: 1913) has rated books and scores: [(44229, 6), (60160, 7), (88281, 6)]
Neighbor 3 (User Index: 9877) has rated books and scores: [(49871, 10), (217165, 8)]
Neighbor 4 (User Index: 12173) has rated books and scores: [(12232, 5)]
Neighbor 5 (User Index: 7706) has rated books and scores: [(2, 9), (377, 9), (458, 9), (573, 5), (574, 7), (578, 7), (586, 6), (834, 6), (986, 8), (1076, 10), (1119, 5), (1120, 5)]
Results saved to neighbor_nonzero_ratings_with_scores.txt
Recommended Books:
ISBN: 1888306149, Predicted Rating: 8.0000, Frequency: 2
ISBN: 0061095540, Predicted Rating: 7.5027, Frequency: 2
ISBN: 1576734013, Predicted Rating: 6.5034, Frequency: 2
ISBN: 0312044194, Predicted Rating: 6.0366, Frequency: 2
ISBN: 0312966091, Predicted Rating: 10.0000, Frequency: 1
```

تحلیل نتایج:

• شباهت پیرسون:

- امتیازهای شباهت پیرسون به طور قابل توجهی بالاتر از هر دو معیار کسینوس هستند. این نشان می‌دهد که پیرسون به خوبی روابط خطی بین امتیازهای کاربران را شناسایی می‌کند و به خوبی تمایلات کلی را در نظر می‌گیرد.
- شباهت‌های بالای حدود ۰,۷۵ نشان‌دهنده‌ی این است که همسایگان الگوهای امتیازدهی مشابهی دارند که باعث می‌شود توصیه‌ها بر اساس سلیقه‌های مشترک قابل اعتمادتر باشند.

• کسینوس و میانگین محور:

- نتایج شباهت برای کسینوس و کسینوس میانگین محور بسیار مشابه هستند و هر دو شباهت‌های کمتری نسبت به پیرسون دارند. این انتظار می‌رود زیرا شباهت کسینوس به زاویه بین دو وکتور توجه می‌کند، در حالی که کسینوس میانگین محور برای میانگین امتیازها تنظیم می‌شود.
- مقادیر پایین شباهت (حدود ۰,۱۷) نشان می‌دهد که در حالی که کاربران ممکن است برخی سلیقه‌های مشترک داشته باشند، الگوهای کلی آن‌ها به طور قابل توجهی متفاوت است. این معمولاً در داده‌هایی که امتیازدهی کاربران به صورت یکنواخت توزیع نشده است، رخ می‌دهد.

کتاب‌های پیشنهادی:

• کتاب‌های پیشنهادی از طریق شباهت پیرسون:

- کتاب‌های پیشنهادی در اینجا امتیازهای بالاتری دارند که نشان می‌دهد همسایگانی که با استفاده از شباهت پیرسون شناسایی شده‌اند، با سلیقه کاربر بیشتر هم‌راستا هستند.
- کتاب‌ها دارای فراوانی هستند که نشان می‌دهد چقدر این کتاب‌ها در بین همسایگان پیشنهاد شده‌اند.

• کتاب‌های پیشنهادی از طریق کسینوس و کسینوس میانگین محور:

- توصیه‌ها از هر دو معیار کسینوس امتیازهای کمتری را نشان می‌دهند که منعکس‌کننده‌ی نتایج پایین‌تر شباهت هستند.
- نتایج مشترک در کتاب‌های پیشنهادی بین کسینوس و کسینوس میانگین محور نشان می‌دهد که هر دو روش نتیجه‌گیری مشابهی در مورد سلیقه‌ها دارند، اما این نتایج به سلیقه‌های کاربران به طور قوی مرتبط نیستند.

نتیجه‌گیری:

• چرا پیرسون عملکرد بهتری دارد:

- روش پیرسون روابط را به طور مؤثرتری شناسایی می‌کند و می‌تواند روابط عمیق‌تری بین الگوهای امتیازدهی کاربران را نشان دهد که این امر آن را به انتخاب بهتری در این زمینه تبدیل می‌کند.

• چرا نتایج کسینوس و کسینوس میانگین محور مشابه است:

- هر دو معیار کسینوس بیشتر بر روی زوایای وکتورهای امتیازدهی تمرکز می‌کنند تا بر روی اندازه آن‌ها، که می‌تواند به نتایج مشابهی منجر شود، به‌ویژه در مواردی که کاربران تمایل دارند در محدوده‌ی باریکی از امتیازها امتیازدهی کنند.

این تحلیل اهمیت انتخاب معیار شباهت مناسب را بر اساس زمینه سیستم توصیه و ویژگی‌های داده‌ها تأکید می‌کند. اگر امتیازدهی کاربران به طور متفاوت توزیع شده باشد، روش‌هایی مانند پیرسون ممکن است نتایج معنادارتری نسبت به معیارهای مبتنی بر کسینوس ارائه دهند.

پیشنهادهای آینده برای بهبود:

پیشنهادهایی که می‌توان برای توسعه‌ی سیستم توصیه‌گر ارائه داد شامل موارد زیر هستند:

1. Matrix Factorization و مدل‌های هیبریدی: تجزیه ماتریس و ترکیب روش‌ها.
2. Cosine Similarity پیشرفته‌تر: استفاده از مدل‌های مبتنی بر شباهت با تنظیمات بیشتر مثل Mean-Centered Cosine.
3. Deep Learning: استفاده از مدل‌های یادگیری عمیق مانند Autoencoders و NCF.
4. Graph-Based Models: مدل‌های مبتنی بر گراف برای داده‌های پیچیده‌تر.
5. Personalized Ranking (BPR): رتبه‌بندی شخصی‌سازی شده برای بهبود تجربه کاربری.
6. Sequence Models: یادگیری از توالی داده‌ها با RNN و Attention.
7. Context-Aware Recommendations: استفاده از اطلاعات زمینه‌ای برای پیشنهاددهی.
8. Regularization Techniques: جلوگیری از Overfitting برای بهبود دقت مدل.

هر کدام از این روش‌ها بسته به نوع داده‌ها، نیازهای تجاری و اهداف پروژه می‌توانند مؤثر باشند. اگر داده‌های شما گسترده و متنوع باشند، پیشنهاد می‌شود روش با مدل‌های هیبریدی شروع شده و سپس از مدل‌های عمیق‌تر استفاده شود تا به دقت بیشتری رسید.

Theory questions:

Q1:

Myung, I. J. (2003). "Tutorial on maximum likelihood estimation". Journal of Mathematical Psychology, 47(1), 90-100.

<https://www.youtube.com/watch?v=XepXtl9YKwc>

<https://www.youtube.com/watch?v=1R34AAHa-2U>

<https://www.youtube.com/watch?v=bXGjQnpGGlo>

<https://www.youtube.com/watch?v=66FqSpf1trA>

Q2:

<https://www.youtube.com/watch?v=QKA4HNEw3aY>

<https://www.youtube.com/watch?v=CfR5WxmZ0I0>

<https://www.youtube.com/watch?v=blwlZloUz-Q>

Practical questions:

Q1:

https://www.w3schools.com/python/pandas/pandas_json.asp

<https://www.geeksforgeeks.org/seaborn-kdeplot-a-comprehensive-guide/>

<https://www.geeksforgeeks.org/how-to-make-histograms-with-density-plots-with-seaborn-histplot/>

Q2:

<https://www.youtube.com/watch?v=OS9xRGKfx4E\>

<https://www.khanacademy.org/math/statistics-probability/modeling-distributions-of-data/z-scores/v/ck12-org-normal-distribution-problems-z-score>

<https://www.youtube.com/watch?v=T0IJT6dDt3c>

Q3:

<https://www.youtube.com/watch?v=6mGMBipt7kU>

<https://www.youtube.com/watch?v=qEur4Xi8dks>

<https://www.geeksforgeeks.org/user-based-collaborative-filtering/>

https://www.youtube.com/watch?v=JJnIR_S6dg0

<https://www.youtube.com/watch?v=kccT0FVK6OY>

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). "Item-based collaborative filtering recommendation algorithms."

Herlocker, J. L., Konstan, J. A., Borchers, A., & Riedl, J. (1999). "An algorithmic framework for performing collaborative filtering."

Jamali, M., & Ester, M. (2010). "A matrix factorization technique with trust propagation for recommendation in social networks."

Desrosiers, C., & Karypis, G. (2011). "A comprehensive survey of neighborhood-based recommendation methods."

Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*.

Ricci, F., Rokach, L., Shapira, B., & Kantor, P. B. (2011). *Recommender Systems Handbook*.